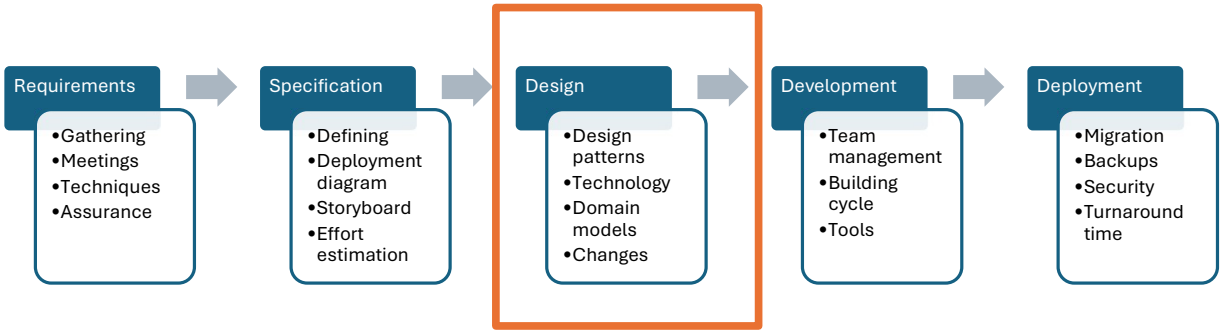



# Design

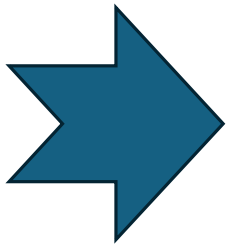
The Software Engineering Process: Phase 3

# What is Design?

- Transitioning away from focusing on the needs of the client, to a focus on **planning** the **engineering process**, specifically:
  - The program, and
  - The development strategy (building the program)

# What goes into a design

- Software Design
  - Domain Model and Design Patterns
  - Critical sections and UML
  - Performance identification areas and design
  - Software development cycle selection and setup
- Software Construction
  - Software development tools
  - Development environment
  - Staging environment
- Quality Assurance
  - Testing procedures, performance considerations, & user experience
- Deployment Procedures

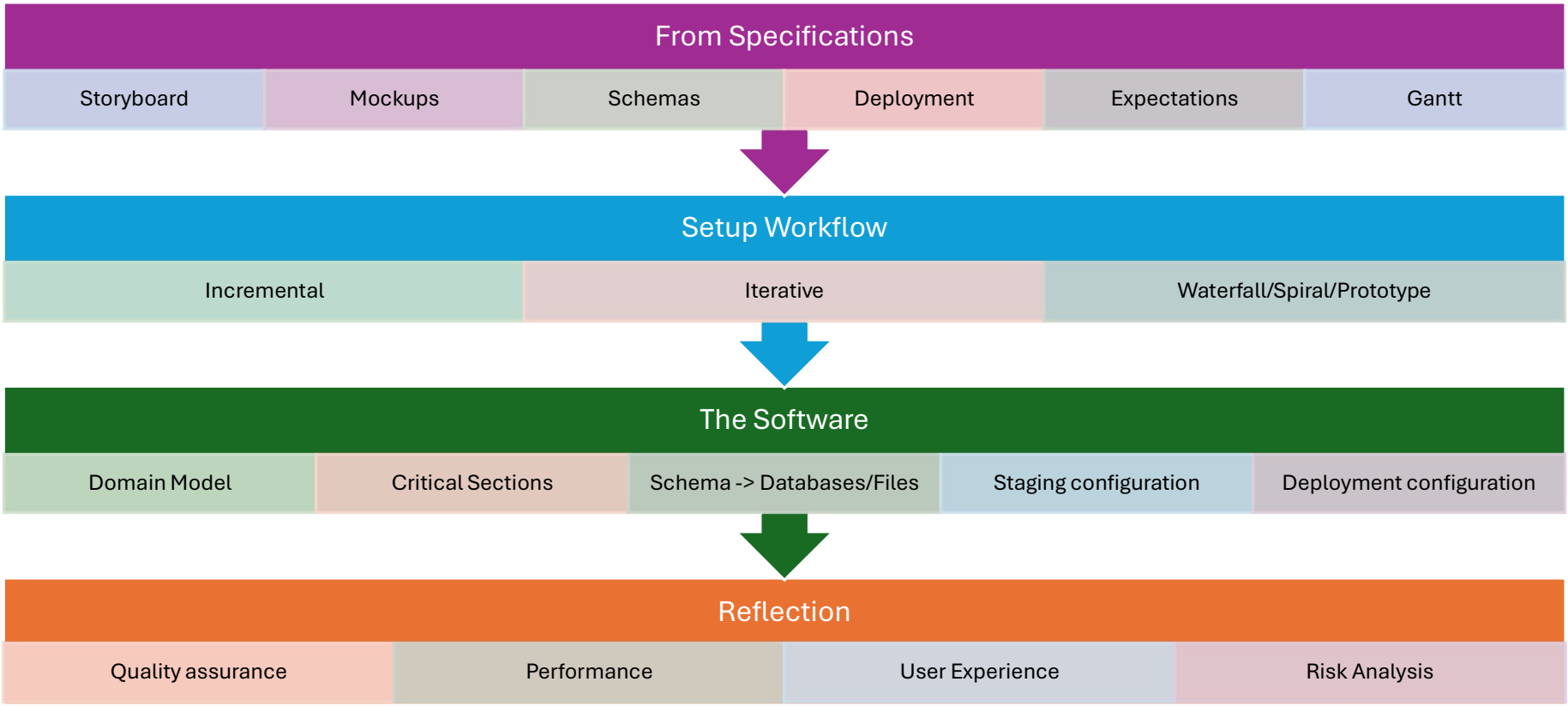


Design Document

# Capturing the Big Ideas ~ Agile the Details

- Agile the details?
  - Ya, there is only so much we can plan, but
- We need to capture the big ideas to control the chaos
  - Planning is needed
  - Figuring out how we will work together without killing each other
    - Working in a good team is like another family (intentionality, forgiveness, tolerance)
    - Professionalism and quality is assumed (and some trust)

# How to: Steps


From Specifications
Storyboard
Mockups
Schemas
Deployment
Expectations
Gantt
Setup Workflow
Incremental
Iterative
Waterfall/Spiral/Prototype
The Software
Domain Model
Critical Sections
Schema -> Databases/Files
Staging configuration
Deployment configuration
Reflection
Quality assurance
Performance
User Experience
Risk Analysis

# The Design Document

- Title Page
- Contents
- 1 Definitions
  - Development Cycle Determination
  - Engineering must-have procedures , engineering won't/will do, must have technologies, best practices to use
  - What is considered success/failure from the engineering view, engineering constraints & assumptions
- 2 Solution Architecture
  - Deployment Environment (Deployment Diagram, Tech determination, Infrastructure and Run-time Environment)
  - Development Environment (Development Diagram & tool selection: IDE, Repo, Dirs, Tests, Backups)
  - Delivery Environment( Staging, Migration (installation, backups, handoff), Maintenance & support)
- 3 Software Design (Domain Modeling)
  - Storyboard
  - Code structure identification: design patterns, algorithms, data structures, files/databases, inputs/outputs)
  - Type of program: Objects oriented programming or modular programming
  - UML Domain Model
- 4 Database, File, and API Definitions
  - Complete schemas & SQL signatures grouped by storyboard sections
- 5 Critical Sections
  - Risk Analysis
  - Mitigation Algorithms through UML Diagrams and SQL
- 6 Appendix
  - Optional other documentation, references/links, standards, etc.

Follow strict formatting standards

# Title Page & Contents & Resources

| Title Page | Table of Contents |
| --- | --- |

Design Document

Project Name

Date

Author(s)
Company Name

## Contents


1. Definitions . . 1
2. Solution Architecture . 3
3. Software Design. . 6
4. DB, Files, API Definitions. 20
5. Critical Sections . 25
6. Appendix . . 35

# 1. Definitions

- 1.1 Development Cycle
- 1.2 Engineering Procedures
- 1.3 Engineering Constraints and Assumptions
- 1.4 Engineering Success and Failure Statements

# 1.1 Development Cycle

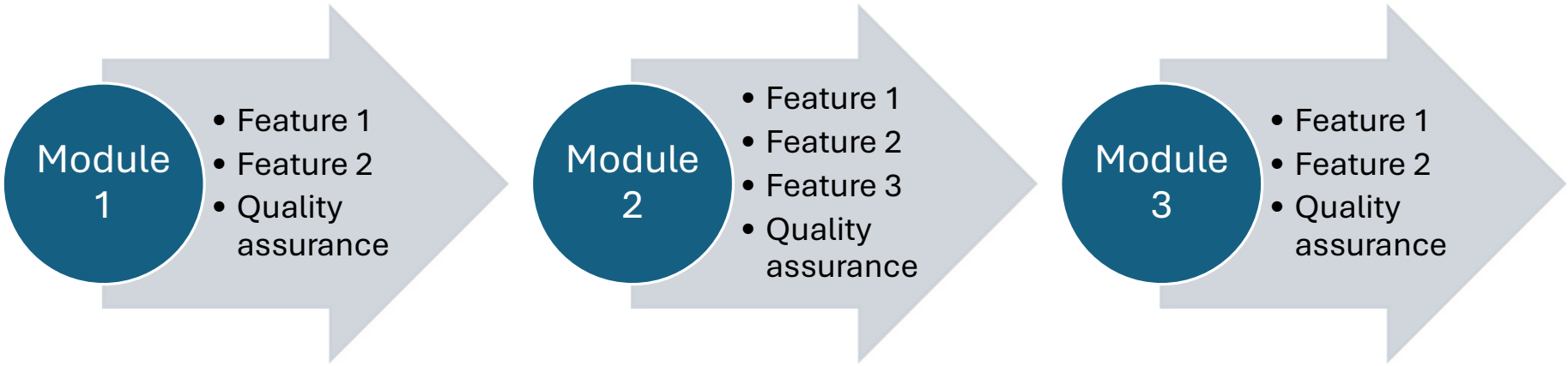
- How will the software be built?
  - Incremental, or
  - Iterative
- How will the team do their work?
  - Standard
  - Paired
  - Spiral
  - Mixed / Custom

# Building Software

- Assume we have the following program:
  - Module 1
    - Feature 1.1
    - Feature 1.2
  - Module 2
    - Feature 2.1
    - Feature 2.2
    - Feature 2.3
  - Module 3
    - Feature 3.1
    - Feature 3.2

# Building Software: Incremental

The software is built up gradually by ordering the modules.



# Building Software: Iterative

The entire application is created at once (all modules are built), but we gradually populate the modules with their features.



# Which workflows are compatible? Why?

- Waterfall
  - Both, but we don't go back after sign-off – Can build in one go.
- Spiral
  - Iterative, but we can repeat a build infinitely
- Prototype
  - Incremental, but we include the client during the QA step

Q: What would each workflow look like for the app?

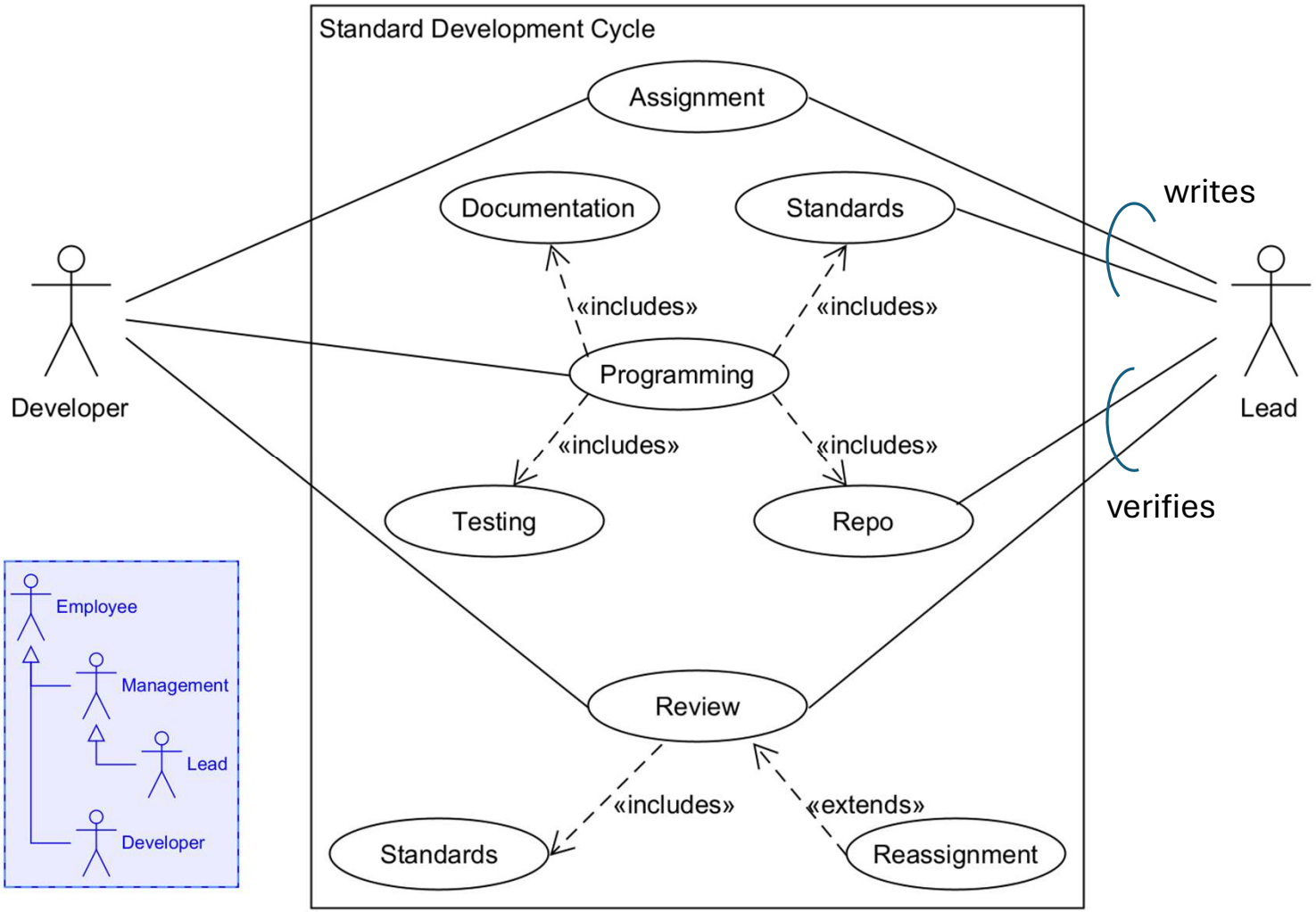
| Questions |
| --- |
| Building a bridge<br>Building a rocket<br>Fancy cooking timer app<br>Upgrading MMO |

# Building software flows

- Spiral
  - Assign unit → Program unit → QA unit → Customer Assessment → Risk & Performance Assessment → Integrate into codebase → QA codebase → repeat unit (poor assessments/QA) or next unit build or redo older unit decision.
- Prototype
  - Define first increment build of complete application → program → QA testing → Run on customer's machines with customer → Evaluate success → Repeat increment (improve success measures) or next increment build decision
- Waterfall
  - Assign unit → Program unit → Integrate into codebase → QA application → Customer assessment → Customer signoff & move to next unit or fix

# Working in a team

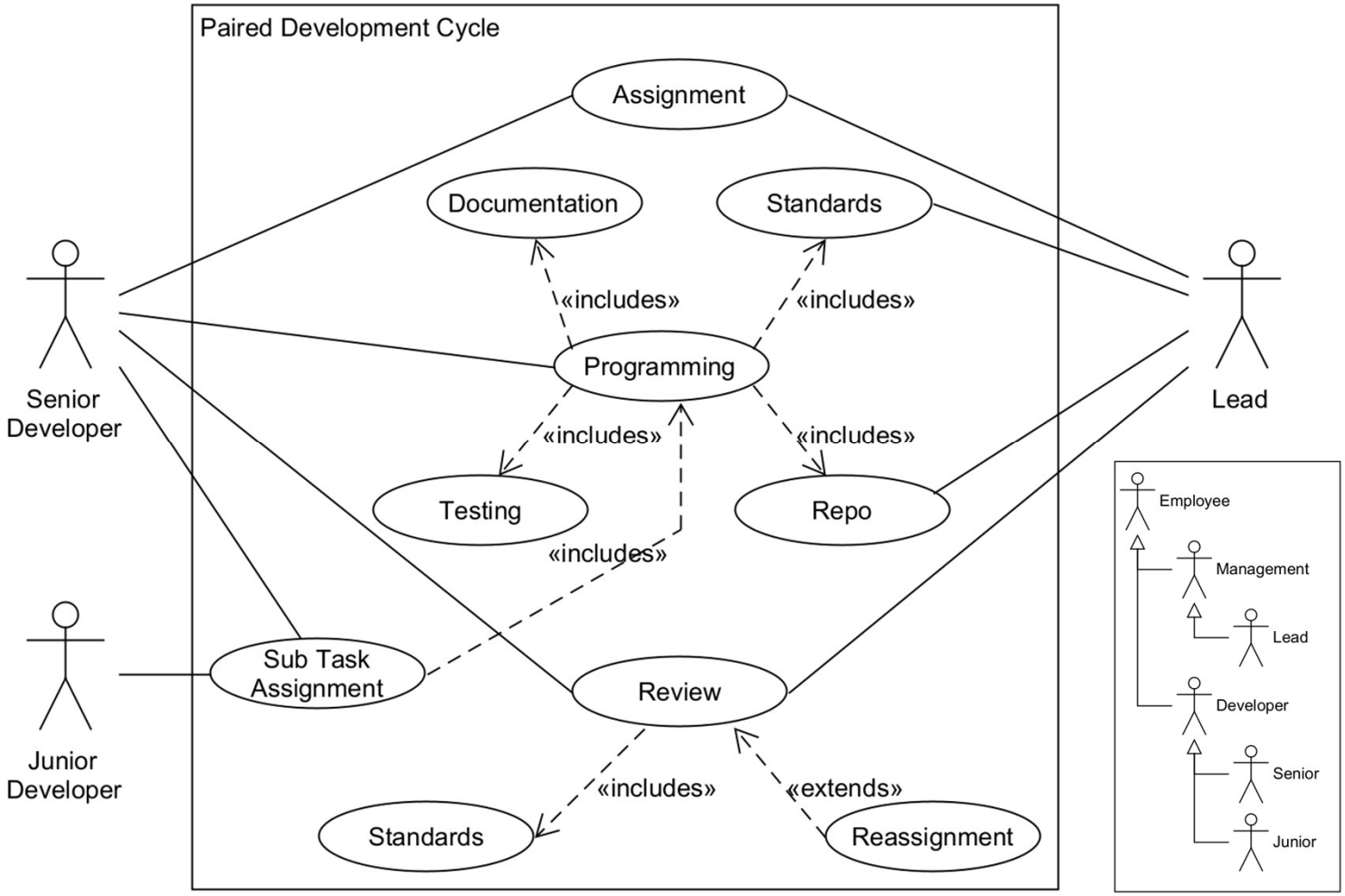
- Standard
- Paired
- Spiral
- Mixed







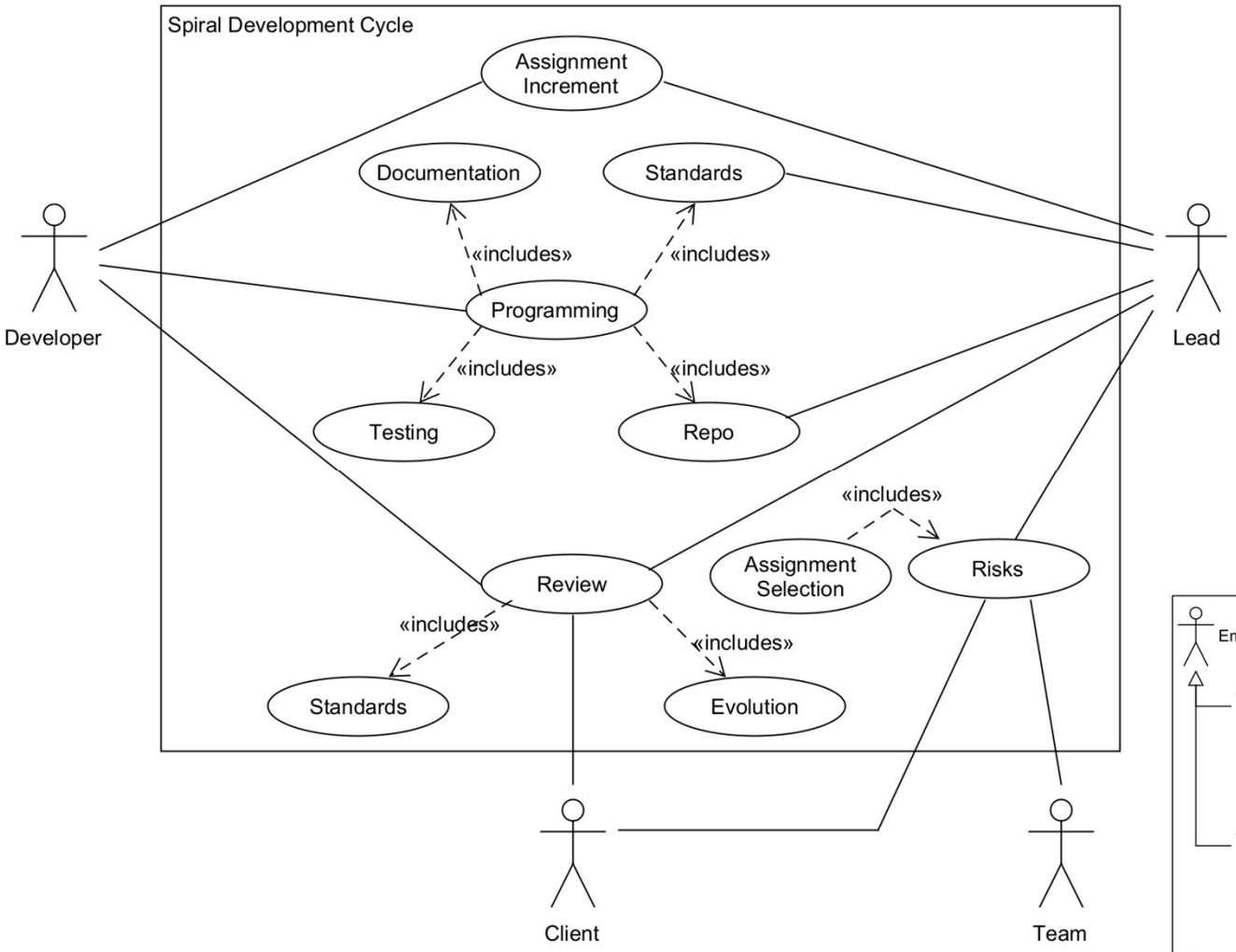
1. Lead writes and assigns work to a developer
2. Developer creates the code while keeping an eye on the specifications and design document
3. The lead reviews the quality of the code with the specifications and design and determines weather developer can be assigned a new task, or the task is not complete.

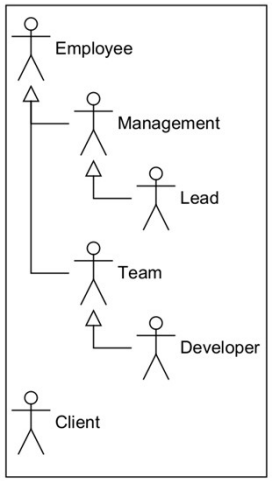






- Similar flow to standard
- Adds:
  - Training of junior
  - Senior assigns subtasks to junior
  - Developing in a group
- Team =
  - Lead +
  - N pairs




1. Risk analysis and step selection with everyone.
2. Detailed assignment increment written
3. Work assigned to developer
4. At review the client is included, which normally results in repeating steps 1-4 until satisfied.
5. If we happen to be satisfied at the review, we evolve (select the next thing to do) and go to step 1.

# Mixed Development Cycles (custom)

- Any of the development cycle structures can be mixed to create your own development cycle.
  - This may include adding your own elements

- Because each project and team and client are different.
  - Get the business done – standard cycle
  - Risky situation – spiral cycle
  - Culture and large teams – paired cycle

# Example

- What development cycle should we use in this situation?
  - We have 7 developers in the team:
    - 1 Lead in Montreal
    - 3 Developers in Toronto
    - 3 Developers in Hong Kong
  - Client is the Canadian Space Agency
  - We are adding a feature to the Canadarm

Big project flow
- Incremental vs Iterative
- Waterfall
- Spiral
- Prototype

Team workflow
- Standard
- Paired
- Spiral

# 1.2 Engineering Procedures

- Defining the standards you will be following when developing, i.e.
  - Daily scrum meetings
  - Regular client demos or scheduled client demos
  - Create a backup at each successful phase of the running program
  - Use a repo, IDE, AWS, Zoom, Slack
  - Common directory structure
  - You push/merge to/from repo, you fix the repo (even other people's code)
  - Morning announcements
  - Using assignments with due dates
  - Regular Gantt Chart progress meetings
  - Etc.

# Example procedure

**Start increment**

- Meet to talk about the increment
- Assign work and discuss due dates (Gantt chart)

**Building (daily)**

- 9 AM daily scrum (how's it going, who needs help, morning announcements)
- 10 AM, everyone pulls repo
- Programming & unit testing
- 4 PM, everyone pushes and resolves repo conflicts, then goes home.

**QA**

- Testing, performance testing, and user experience testing (after the programmer has completed the feature)
- Pass (go to next increment) or Fail (fix the increment)

**End Increment**

- Meet to talk about the current state of the increment

# Example procedure

**Start increment**

- Meet to talk about the increment
- Assign work and discuss due dates (Gantt chart)

**Building (daily)**

- 9 AM daily scrum (how's it going, who needs help, morning announcements)
- 10 AM, everyone pulls repo
- Programming & unit testing
- 4 PM, everyone pushes and resolves repo conflicts, then goes home.

**QA**

- Testing, performance testing, and user experience testing (after the programmer has completed the f
- Pass (go to next increment) or Fail (fix the increment)

What should programmers be doing while this is happening?

**End Increment**

- Meet to talk about the current state of the increment

# Example 2:

- Monday: plan for the week, with Friday deliverable per person.
- Daily scrum at 9 AM.
- Morning: Pull from repo.
- Day: programming.
- End of day: push to your branch.
- When deliverable completed: merge branch to main & debug main.
- Friday PM: Review of week, suggest changes to workflow, etc.
- Milestone completed: demo to client & record issues.

# Question

- What procedures would be good in the Canadarm example?

# 1.3 Engineering Constraints and Assumptions

- Constraints
  - Things that will constrain the development choices
    - Languages, tools, hardware, technology
    - Funding, timing/schedules, people, etc.
- Assumptions
  - Things that are obvious to you, but if they happen, would impact the project.
    - Developer expectations of the client
    - Developer expectations of suppliers
    - Developer expectations of programmers, etc.

NOTE:
List only things that are concerns.

Do one for Constraints and another one for Assumptions.

# Format

| Item | Constraint | Expected Impact |
|---|---|---|
| | | |
| | | |

| Workaround Plan |
|---|
| |

# Example

- Constraints
  - Our company goes on vacation in July, which is in the middle of the project
  - Key people are currently busy on another project, expected back in February.
  - Project funding is insufficient, needs $15K, provided 10K.
  - Need to learn backend database programming
- Assumptions
  - Client will be available for Spiral – does not delay the project
  - Asked for resources will be available for January 21st

# 1.4 Engineering Success and Failure Statements

- Success Statements
  - What are the goals, as a developer, you would like your team (including yourself) to meet?
  - In relation to the Gantt chart, what goals would you like to meet?
  - In what ways would you like this project's successes improve compared with your previous project? Etc.
- Failure Statements
  - If X happens, you would not be happy with yourself, your team, or the project.

# Format

Do one for Success and another one for Failures.

| Item | Success | Expected Impact |
|---|---|---|
| | | |
| | | |

| Discussion & Alternatives |
|---|
| |

# Example

- Success
  - Meeting all the client delivery dates.
  - We did not go over budget this time.
  - Client is happy, etc.
- Failure
  - Delays due to poor scheduling or monitoring of others.
  - Failure of team synergy exercises and basic teamwork dynamics.
  - Poor documentation, state of the repo, etc.

This is useful during your regular project reviews. These become harbingers. When you see them arising, it may be time to start planning how to deal with them.

# 2. Solution Architecture

These are mainly Deployment Diagrams, each focuses on different parts of the development process.

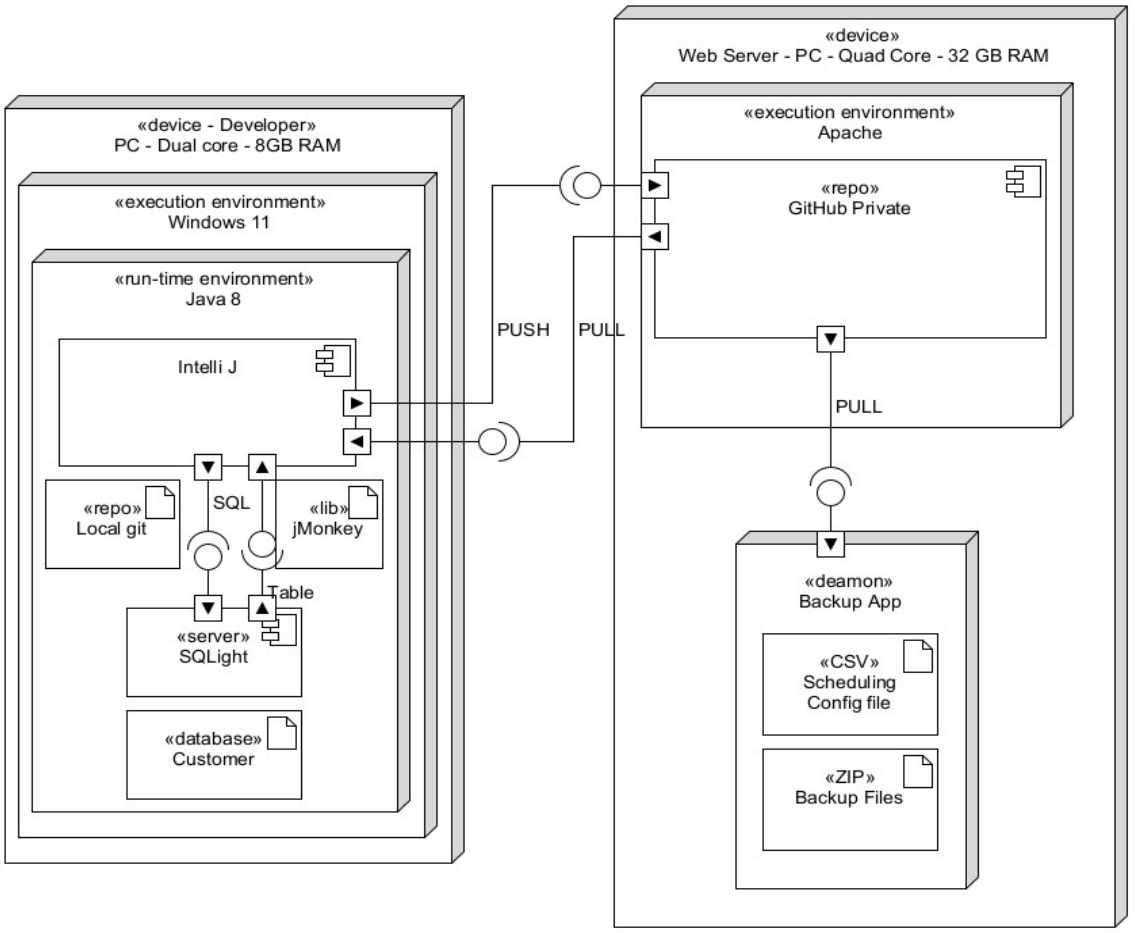
- 2.1 Development Environment
  - The tools and infrastructure the programmers will be using
- 2.2 Staging Environment
  - The run-time environment where the QA people and client will review the program.
- 2.3 Deployment Environment
  - A description of the client's run-time environment.
  - Configuration and deployment strategy (procedure, human & script).
- 2.4 Evolution Environment
  - How will the program be supported (bugs, help, training)?
  - How will the program be upgraded over time (requirements, procedure, install)?
  - How will the program be retired (delete/archive, data migration)?

# 2.1 Development Environment

- Important to standardize the work environment between developers.
  - Different OS, different IDE, etc., adds complexity when merging code.
  - Many employers often:
    - (A) Buy everyone the same brand of computer to work on, or
    - (B) Tell everyone to SSH into the same server, developing on that server, or
    - (C) Use Docker to create a common environment.
- When planning, we need to define the developer's working environment.
  - Deployment Diagram (for configuration), a list (for completeness), and procedures (expected/optimal way to do things).

# Example – Basic development environment

- What does this imply about the workflow?
- What technical knowledge do we need?
- What needs to be installed?



# List

Incorporates: iterative vs incremental, waterfall/spiral/prototype, pair-programming/classical/spiral-programming.

| Item | Name | Version | Description/Environment | Quantity |
|---|---|---|---|---|
| | | | | |
| | | | | |

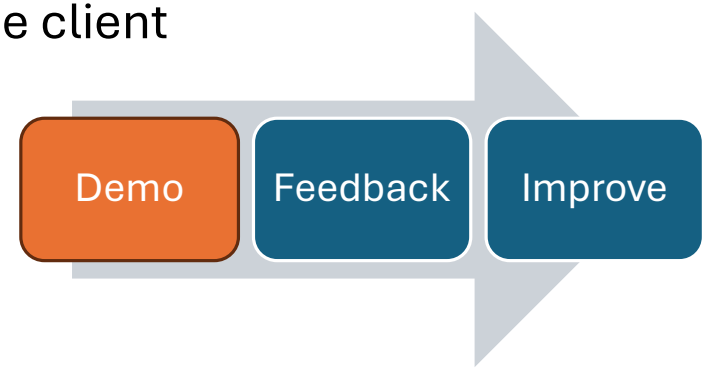
| Installation Procedure |
|---|
| |

| Expected Workflow |
|---|
| |

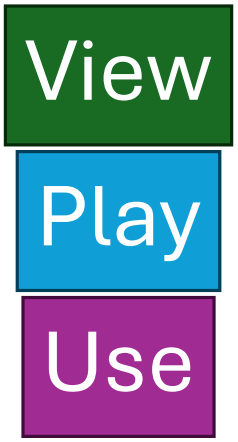
# 2.2 Staging Environment

- The run-time environment you will use to simulate the client's deployment environment.
  - Deployment disturbs the client's company
  - You will need to "deploy" often for integration testing and compatibility testing
  - We need a fake deployment environment
  - You need a safe space to demo progress to the client



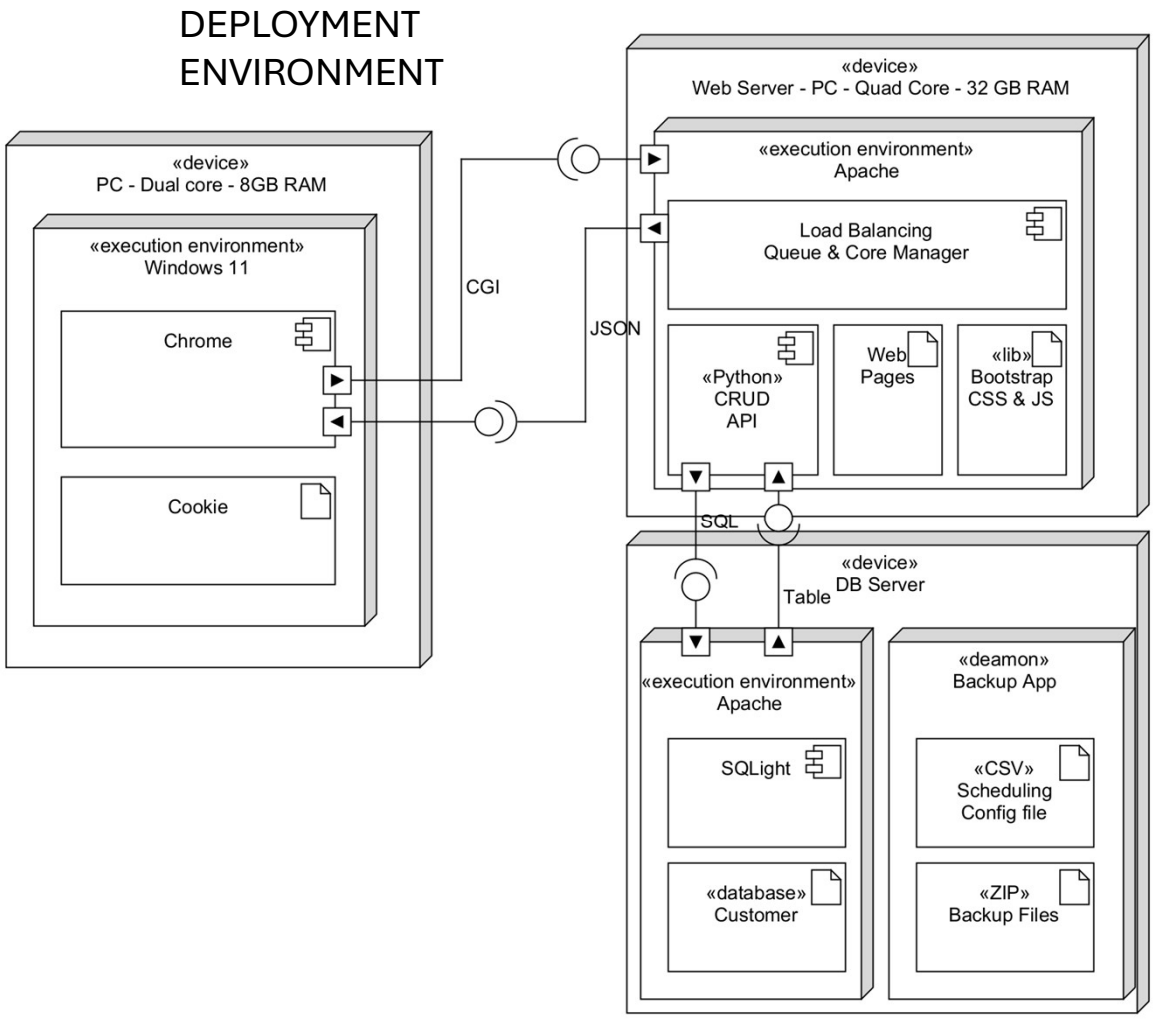
# Types of staging




- Integration testing and client validation:
  - Developer location –
    - Invite the client to your office and show them the program. They might touch it.
  - Running on the developer's laptop –
    - Install on laptop. Visit the client. Show them the program. They might touch it.
  - Running on client's laptop –
    - Visit client. Install on their laptop. Show them how to use it. Leave it with them for a few days.
  - Installing on a server that the client can access –
    - Build a Staging server. They "play" for a few days. Data is deleted.
  - Installing it on the deployment environment –
    - Client starts to use the prototype program, maybe even in production. Data concerns.

# What is staging?

- How should the Staging environment look like, given this deployment diagram?



# Question

- What would be a good staging environment for:
  - A website?
  - The Canadarm example?
  - An online store that needs to start selling product?

# What is staging? - caveat

- Since the staging environment is not exactly like the deployment environment, this means we may not find all the bugs – those bugs related to the particularities of the deployment architecture.
- How should we handle the testing after we deployed the program?



# About Software Deployment

“Many security problems arise because the system is not configured correctly when it is deployed in its operational environment. You should therefore always design your system so that facilities are included to simplify deployment in the client’s environment and to check for potential configuration errors and omissions in the deployed system.” – Ian Sommerville

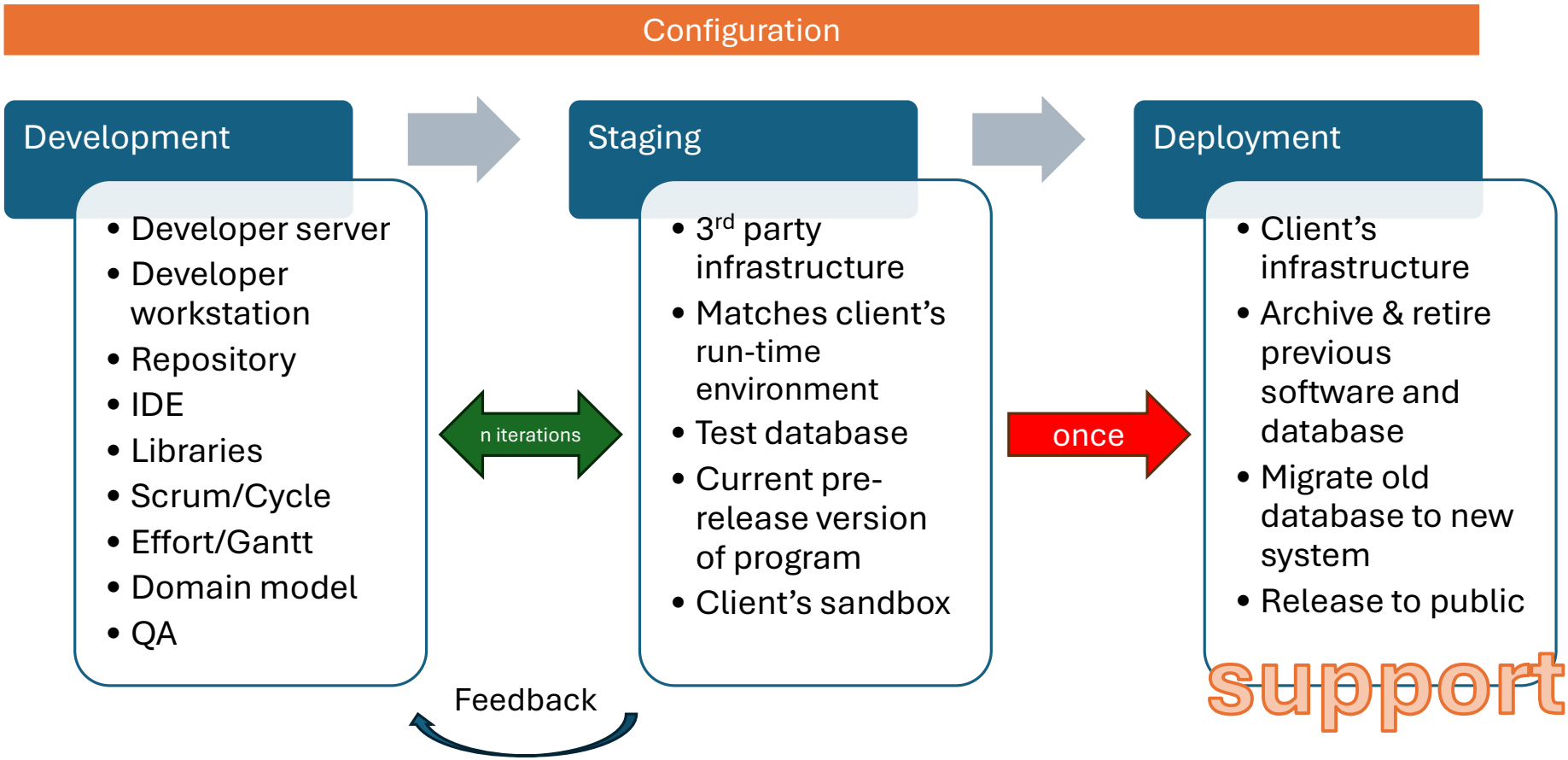


# Mismatch Areas

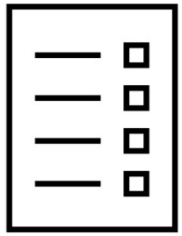
- System
  - Different operating system
  - Different system resources, like DLLs, installed OS Components
  - Different security settings
  - Different hardware
  - Different load reality between client's site and your development site
- Communication
  - Client uses the program in unexpected ways

# About Software Deployment

**CONFIGURATION**: All the adjustable elements of a run-time environment (Installed hardware, OS services, Tools, Libraries, configuration files, etc.)



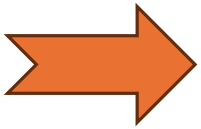
# Best Practices



Identify the client's run-time configuration

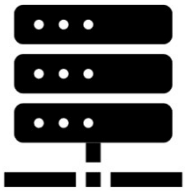


Identify the configuration needs of the application

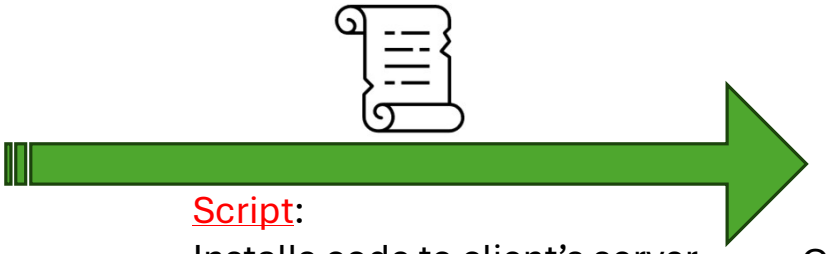




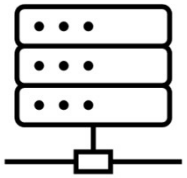
Identify the differences



Staging server:
Merge of client's run-time and what application requires.



Script:
Installs code to client's server and configures the server with the identified differences.



Client's server

# Example

**Client's Environment**

- Windows 10
- Printer queue active
- Wi-Fi, Internet, firewall
- 2-core CPU
- 8 GB RAM
- 4-core GPU with 20 MB

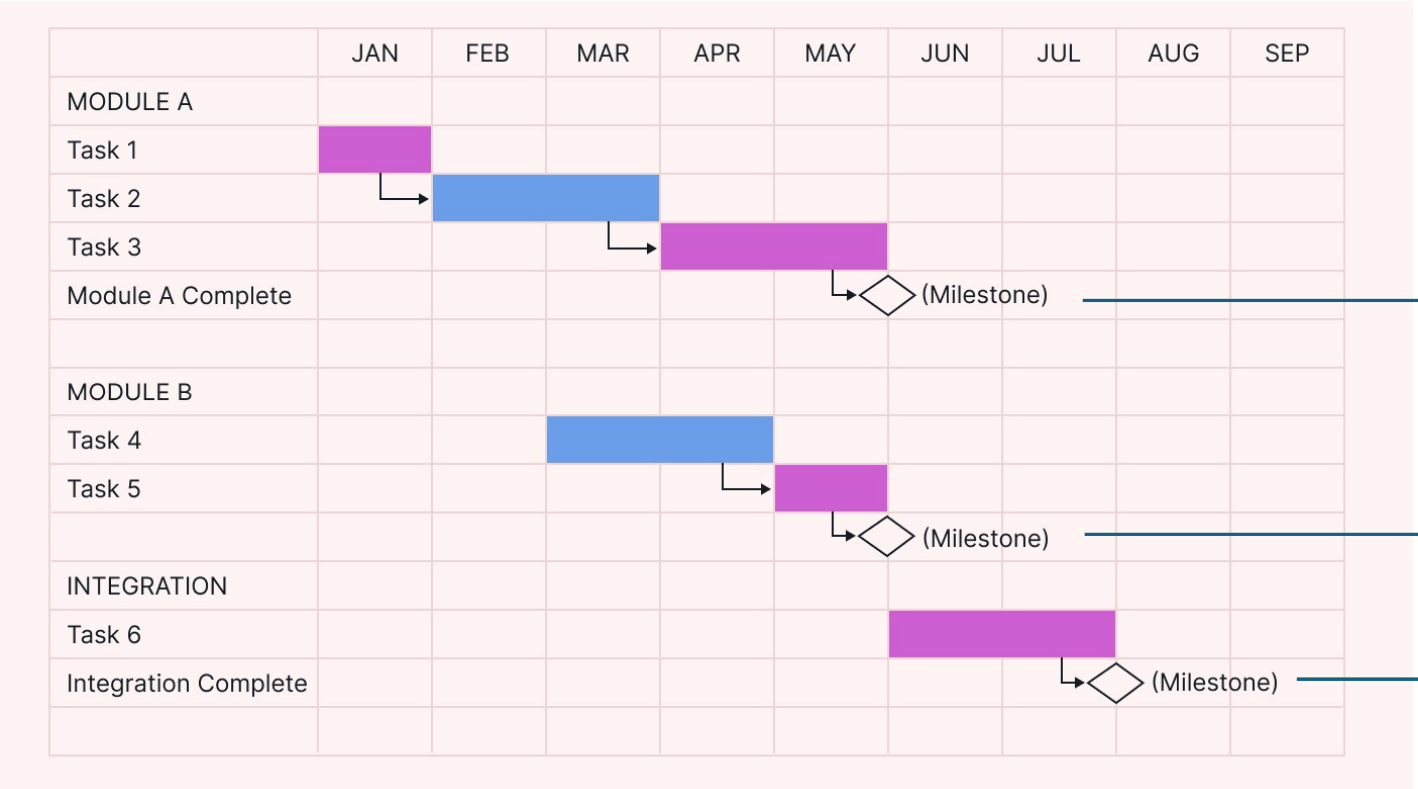
What are the configuration differences?
What should the staging server look like?
What should be the installation procedure?

**Application's Requirements**

- Windows 10 or 11
- Compiled C++ application
- No internet and firewall
- 4 GB RAM
- 4 to 6-core GPU with 8 GB
- Not added to Window's registry
- Invokes Python script

# Usage: The Staging Environment

Simple Gantt chart




QA happens at staging, including configuration testing, and a demo to the client.

Staging

Staging

Staging

https://www.usemotion.com/blog/milestone-chart

# 2.3 Deployment Environment

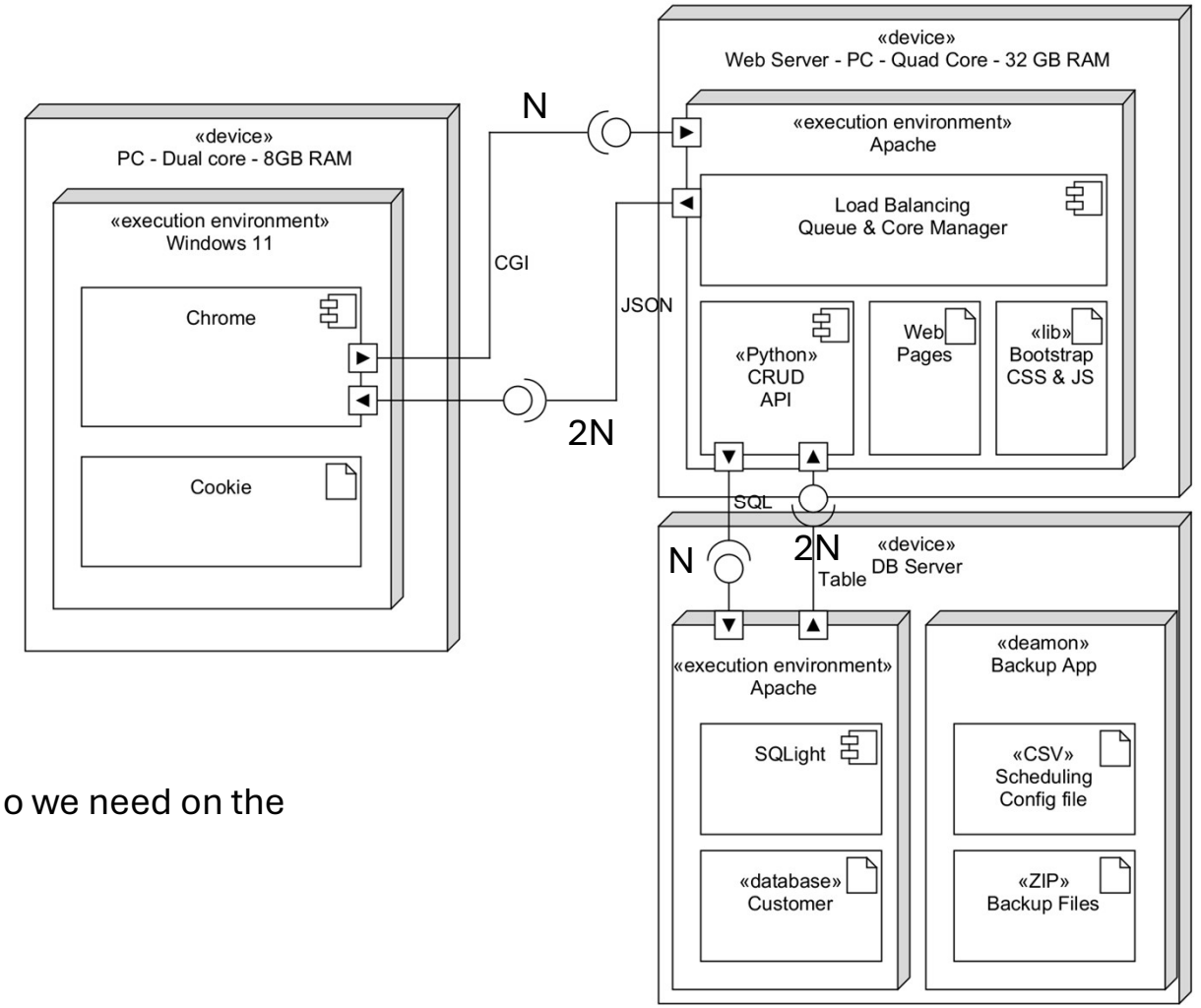
- Define the client's infrastructure:
  - Option 1 – your program must run on the client's infrastructure without modifying the client's infrastructure
    - Only one deployment diagram is needed.
  - Option 2 – the client's infrastructure must be upgraded to run your program.
    - Two deployment diagrams are needed, and an upgrade plan
      - Deployment diagram 1 is the current client infrastructure
      - Deployment diagram 2 is the upgraded client infrastructure
      - The upgrade plan describes an ordered step-by-step plan to upgrade the infrastructure
- Define the Installation procedure
  - How will your program be installed on their infrastructure
    - Includes: Decommission old software, migrate databases to new software

# Is the client's current infrastructure adequate?

Assume they have 100,000 customers who use their service multiple times daily.

Notice the numbering on the arcs. What does it imply?

Who do we need on the team?



# Configuration Steps

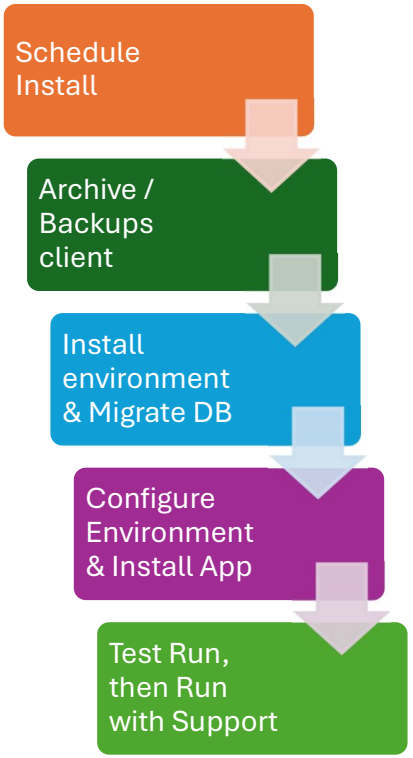
Scripts standardize and allow us to know precisely what happened. They can be updated and improved.




---------- DEPLOYMENT PART 1 ---------- ---------- DEPLOYMENT PART 2 ----------

# Deployment Part 2:
# First Time Installation & Data Migration

- Goal: Install the app from staging to client's environment
- Procedure:
  - Schedule the event with the client.
    - Everyone stops using their computer for that period
  - Archive all data and software that will be retired.
    - Mkdir archive, mv *.oldApp, *.oldData. Remove old configuration.
    - Write a script that can quickly reactivate the archived application, in the worst-case.
  - Install the new database and verify it works through command-line.
  - Migrate archived data into new database, if needed.
    - Write a script to automate this process & verify if this was done correctly through command-line.
  - Configure client's computer to work with the new app and database.
    - Write a script and use your checklist to ensure this is done properly.
  - Install the new program on all client computers
    - Verify that it is running properly with the client.
  - Tell everyone at client's location to start using their computers
    - Hang around to handle issues.
  - Setup a support line for after your team leaves. Be prepared to return in the 1st month.



# Data Migration Example

- Old client database:
  - Customer table: custID, name, email, address.
  - Orders table: orderID, itemDescription, quantity, unitPrice
- New client database:
  - Customer table: custID, name, email, street, city, postal, country.
  - Orders table: orderID, itemID, quantity, subtotal
  - Items table: itemID, unitPrice, quantityInStock , description
- What should we do? How should we migrate?
  - By hand or via script?
    - What would it look like?
  - How should we handle this?

# Install Example

- The client has 5 workstations (4 Windows and 1 Mac) connected to a LAN with an in-house Windows server, and 1 sales rep using a Windows laptop only connected to the company's website that is hosted by a 3rd party. The website does not connect to the in-house server.
- The app is a C++ application that runs on Windows.
  - Various DLLs need to exist on the computer for the app.
  - The database uses SQLight3.
- The staging environment is an AWS Windows instance configured to match well with the client's in-house server and the new app.
- The goal is to host the database on the server and the app locally on the 5 workstations and the sales rep's laptop. The app downloads data locally when not connected to the in-house server.
- Questions: You have just upgraded the app…
  - Given the description, is there anything we should be concerned about?
  - How do we install?
  - Do any procedures need to change on the client's side?

# Basic Install Procedures

- Step 1: Manual procedure
  - Manually configure OS and install non-downloadable apps.
  - Archive old system.
  - Copy new app into a temporary subdirectory, if it cannot be downloaded.
- Step 2: Run the script to do the following:
  - Auto download and install 3rd party apps
  - Automatically configure those parts of OS that can be
    - Mkdir, cp, set VER and ENVIRONMENT variables, make files, cron jobs, etc.
  - Auto install (or sometimes build from scratch script) a new database.
  - Merge archived data with the new database.
  - Run new database validation scripts.
  - Install the new app's launch script (or shortcut)
  - Clean up working files and directories.
- Step 3: Run the program by yourself by hand to see if it looks good (run built-in unit tests)
- Step 4: Run the program with the client by hand to see if it looks good
- Step 5: Give access to the app to everyone and hang around for a while just in case.

# Hotline & Ticketing System

- It is essential to provide a great deal of support for a short period after installing a new app (e.g., one month), then to reduce the intensity of the support over time.
  - 1st month:
    - On sight presence + Hotline phone number
  - After 1st month:
    - Access to the hotline + online ticketing system
    - Your app could even have the ticketing system integrated
      (e.g., opens browser to online ticketing system, or emails to support staff, or built-in ticket form using a remote API to your office ticket database)

# Maintenance

- What is maintenance?
  - (A) Handling complaints from the client after project is completed.
  - (B) Regular updating procedure
  - (C) End of support
- Immediately after project completion
  - There will be complaints, build this in the price & Gantt chart & decide on a procedure.
  - This has a short time limit (e.g., one month)
- After project completion
  - How will they contact you? Your response time should be prompt, brand-new contract.
- Service contracts
  - Regular updates
  - Standard way to send in issues
- A long time later, without a contract
  - Like “after project completion”, but response time does not need to be quick, however there should be a way to file an issue with a pricing plan.
  - Often this is treated like a brand-new contract.

# Example Maintenance Plan

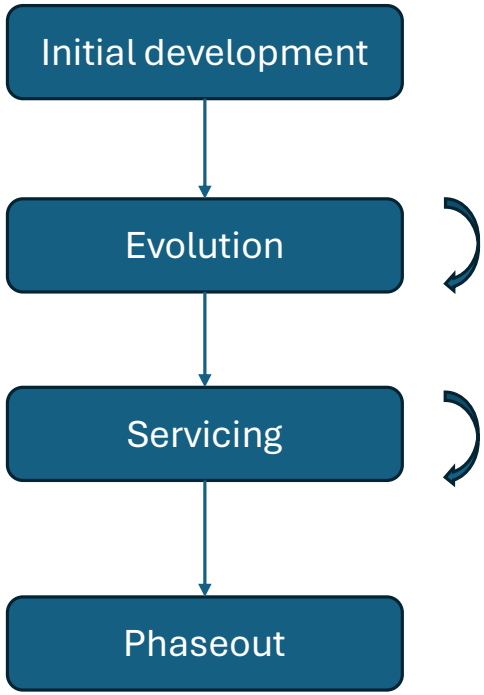
- Setup a ticket system for client to register bugs
  - Call-in, email-in, website
  - Client should be able to view and track progress
  - Should have dates: registered, assigned, review, completed
  - Should have names: client who registered, developer who worked, reviewer
  - Developers and manager have access to assign, edit, and log.
- Process for communicating back to client
  - Email, demo
- Method to update the software at client's location
  - Physical update
  - Automatic update

# Question: maintenance plan

- Website?
- Canadarm example?
- Local desktop accounting system?

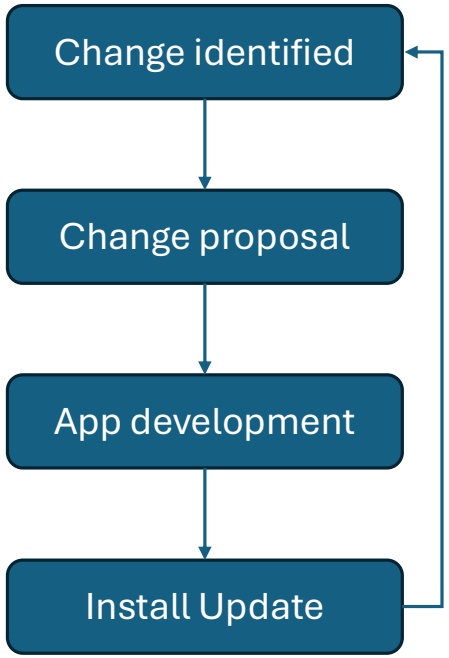
# 2.4 Evolving Environment

- Evolution is that stage in the app's life cycle where the client is still very interested in expanding its capabilities above from what was initially-developed with multiple additional contracts.
- This is different from servicing when no new features are added, instead we are keeping it alive with fixes.



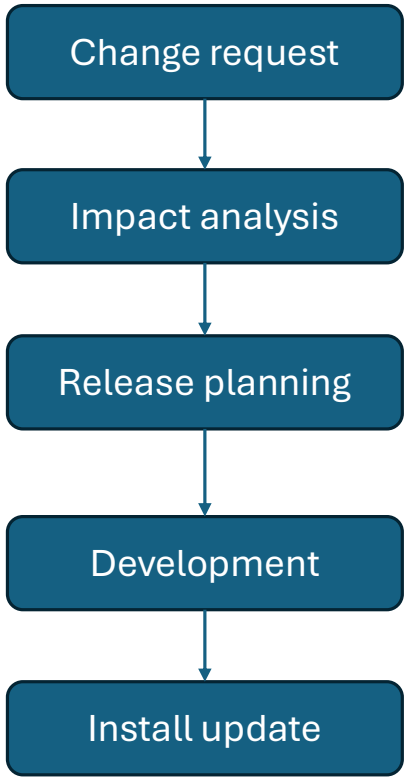
# How does evolution happen?

- The client typically identifies a new need.
- They contact you to create a plan: specifications, effort estimation, Gantt, and cost.
- Once approved, development and updates happen.



# How to update the app?

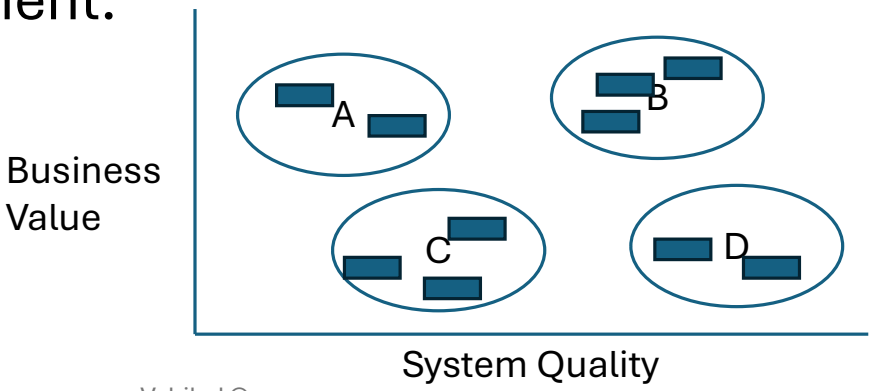
- Impact analysis includes:
  - Fault identification and repair
  - Side-effect mitigation
  - Platform adaption issues
  - Other enhancements
- Release planning
  - When is it best to install?
  - What needs to be shut down to do the install?
  - Plan B?



# Is evolution costly?

- Yes
- First builds are cheaper than evolution or maintenance
  - People forget the source code over time
  - New data structures may not be compatible or ideal with older structures
  - New programmers may not even know the development language
- A legacy system assessment:

**Which should we retire**?
A = High business value low quality
B = High business value high quality
C= Low business value low quality
D = Low business value high quality



# 3. Software Design

- 3.1 Storyboard & Critical Sections
- 3.2 Code Structure Identification
- 3.3 UML Domain Model
- 3.4 Module Breakdown



Top (global view)

Down (detailed developer view)

Notice, no class diagram.

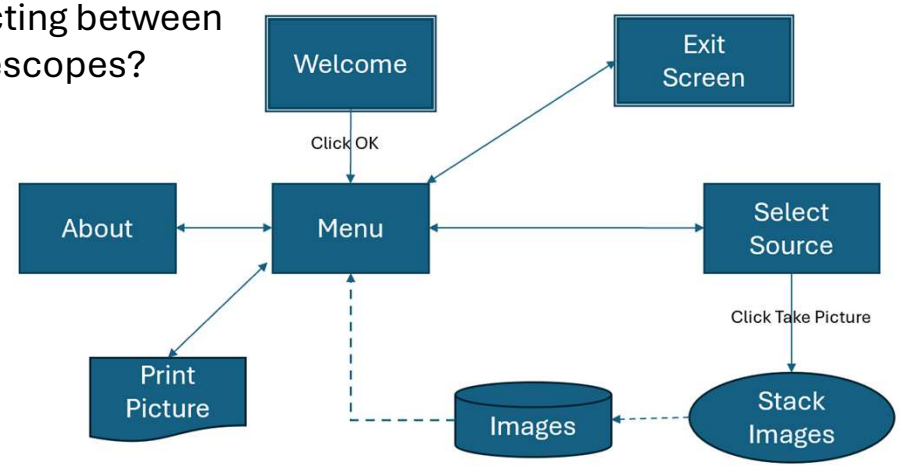
Revisit, reflect, & programmer's point of view

# 3.1 Storyboard and Critical Sections (Vr 2)

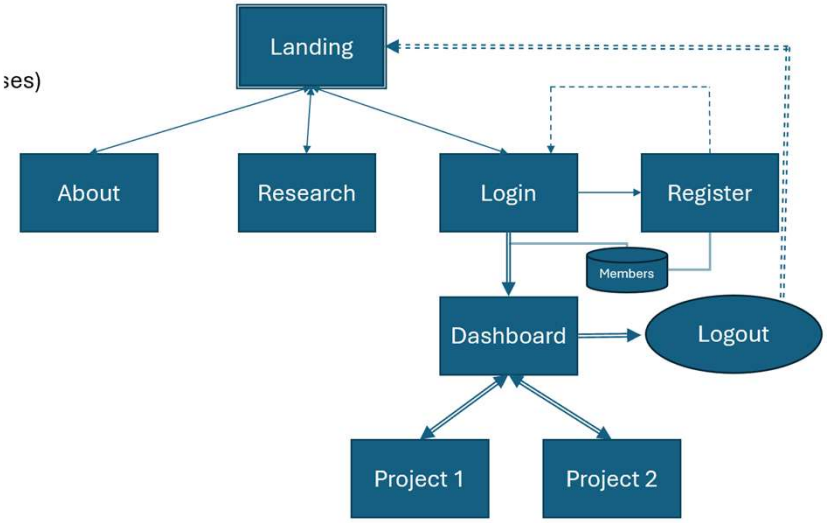
- Inform the developers and revisit the diagram
- How would the storyboard change given the analysis of the deployment, development, and maintenance environments?
- Examples, should we add:
  - Import / Export module?
    - To help with migration and demos
  - Backup module?
  - Security module?
    - Bad guy and security concerns raised during analysis of the deployment diagram
  - Ticketing support?
  - Install/update scripts?

# Storyboard (version 2)

Selecting between
5 telescopes?




Project 2 auto backup?




Purpose 1 – Tells developers about the macro structure of the project.
Purpose 2 – Revisit, reflect, and decide on modifications given deployment diagrams: deploy, develop, stage.
Purpose 3 – Add developer critical sections to the list of client critical sections.

# Question

- Cooking timer app version 1
  - A timer only with dial graphic
  - With three customizable set times stored in a CSV file: timers.csv
  - With the ability to customize the timer layout stored in: layout.csv
- Cooking timer app version 2
  - Add recipe module
  - Add an infinite number of timers
  - Add more than one timer default layout: dial graphic and ordered timer list
- Import/export, backup, security, tickets, install/update, migrate?

# Developer's
# Critical Section Identification Table

| Item | Title | Description | Location | Owner |
|---|---|---|---|---|
| 1 | | | Module | Client |
| 2 | | | Screen | Developer |
| 3 | | | Phase | |

Using the Storyboard and the client's stories, what are all the critical sections? The goal is to take what was discovered from the specifications document and reformat it for developers. Important: In doing this, we do not want to lose the client's perspective.

Question: What are the critical sections from the previous question?

# 3.2 Code structure identification

- Identification of the technologies, Design Patterns, and data structure needed to implement the application.
- This is presented through three tables with some explanatory text to help the reader understand the tables. Each table has its own descriptive text.
  - Table 1 – System-Wide Structures
    - Things that cannot be assigned to a specific module of the program
  - Table 2 – Module-Based Structures
    - Things assigned to a specific module/class (divided into modules/classes)
  - Table 3 – Procedure-Based Structures
    - A table divided into sections based on the phases of the project (not app), like deployment, evolution, development, etc.

# Table format (variable)

- Itemized
- Grouping (System, Module, Procedure)
- Title
- Purpose
- Technology description (language or pseudo-code/data structure)
- Location (e.g., method name, step name)
- Owner (client or developer – who asked for it?)

# Example

- What structures do we need for the cooking timer app?
  - Programming language(s)
  - Libraries
  - Data structures
  - Files / Databases
  - Algorithms
  - System components
  - Network components
  - Hardware components
  - Interrupts
  - Error handling
  - Deployment

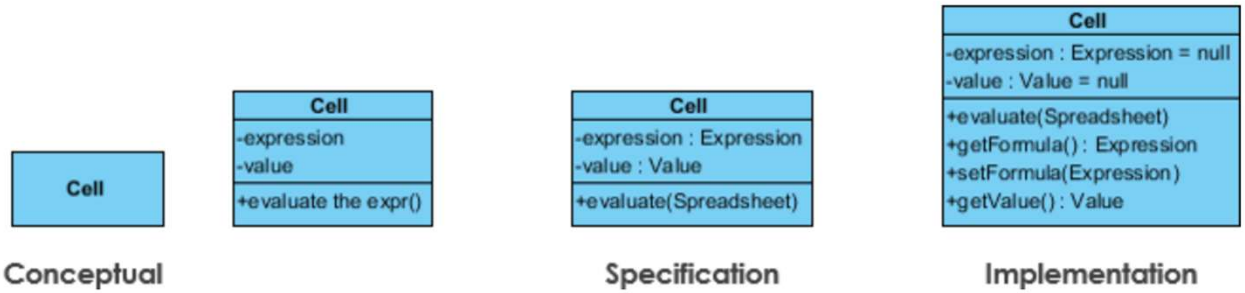
# 3.3 UML Domain Model

- Like the storyboard is a single picture of the project from the user's perspective, the Domain Model is a single high-level picture of the application from a developer's perspective.
  - Think object-oriented programming (or modular programming)
  - Think design patterns
  - Think data structures and abstract data types

The Domain Model combines diagraming techniques from Class Diagrams and Sequence Diagrams.

# Domain Model vs Class Diagram

- Not to be confused with the UML Class Diagram




- A class diagram is a complete definition of all the methods and members of a class.
- Originally, software engineers attempted to define the complete class diagram before building the project. This turned out to be difficult. Now, we instead beuild UML Domain Models which or “object diagrams” and not class diagrams (the distinction is subtle).
- Most importantly, the domain model does not define all the elements of the class, but instead focuses of specific important elements, and assumes that the less important details will be determined later.

# Differences

**Domain Model**

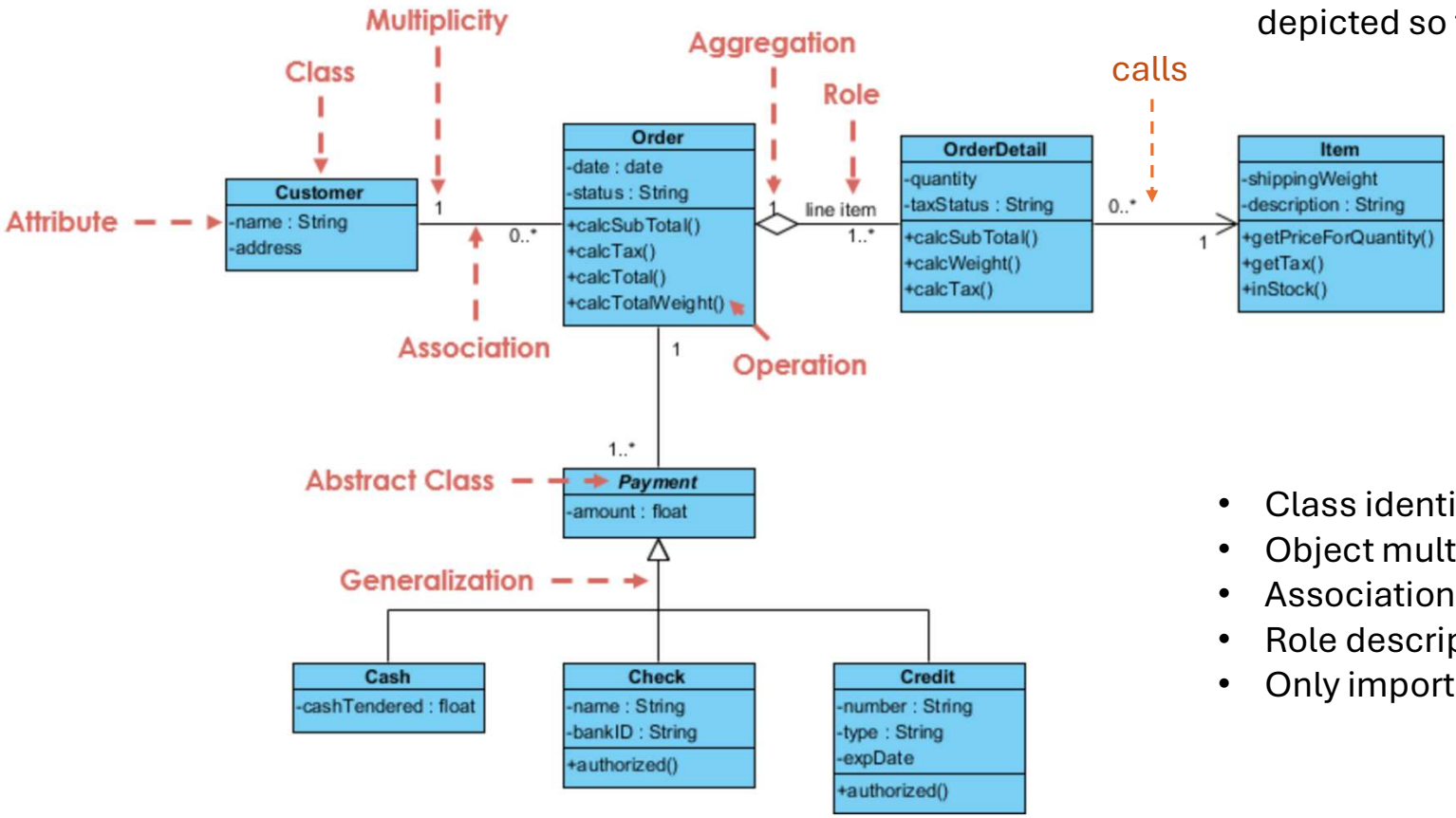
- Class diagram syntax +
- Structural class information
- Contains some flow arrows
- Brief classes
- Some code and conditions
- Notes can be used
- Responsibilities optional
- Arc labeling expected

**Class Diagram**

- Class diagram syntax
- Structural class information
- No flow information
- Detailed classes
- No code or conditions
- Notes should not be used
- Responsibilities required
- No arc labeling

# How is this domain model different from a class diagram?

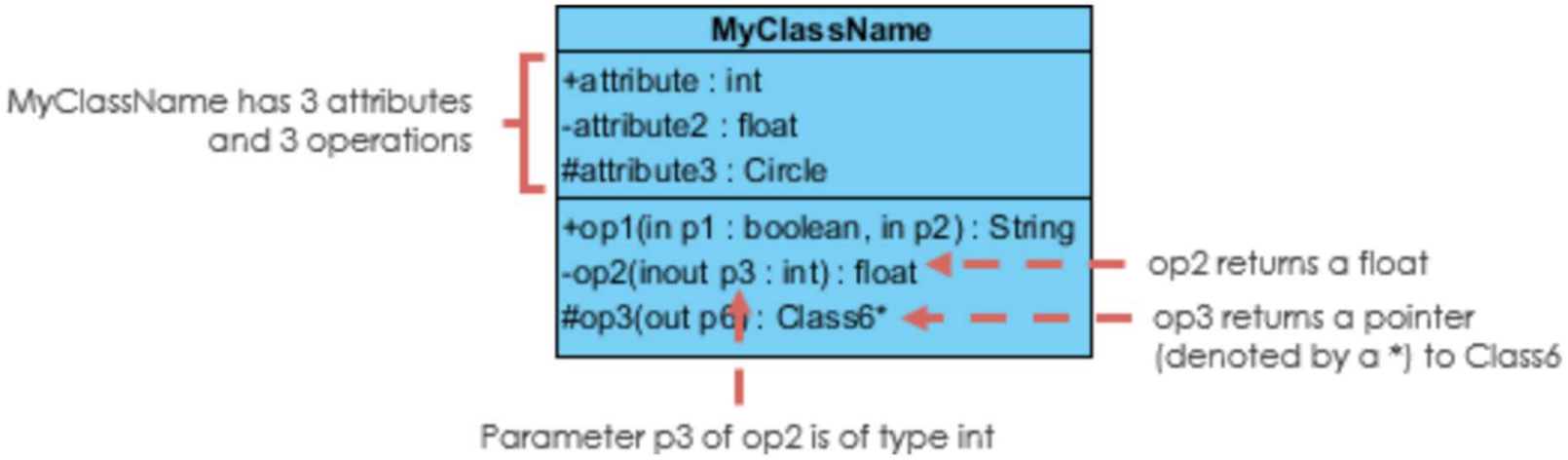
Not too much flow depicted so far...




- Class identification
- Object multiplicity
- Association type
- Role description
- Only important members

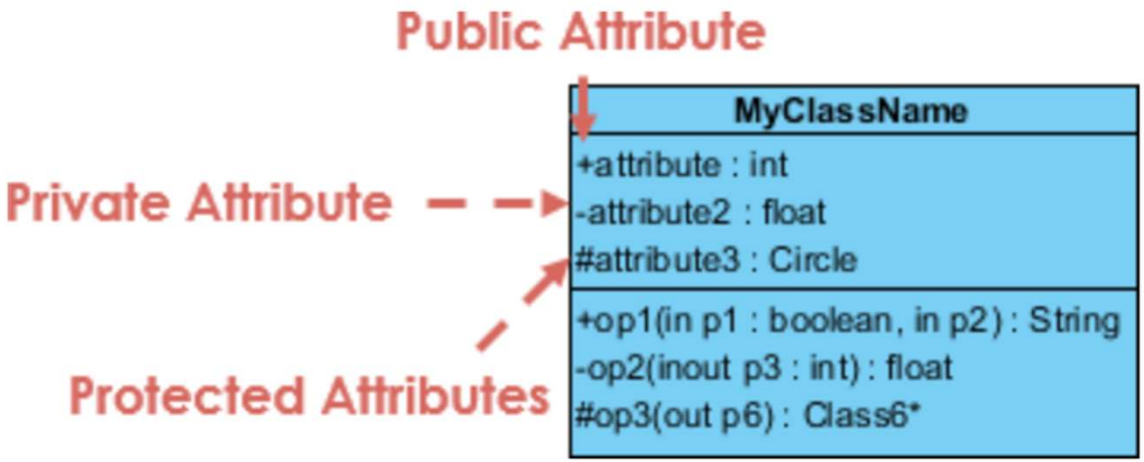
# The class

- Structure



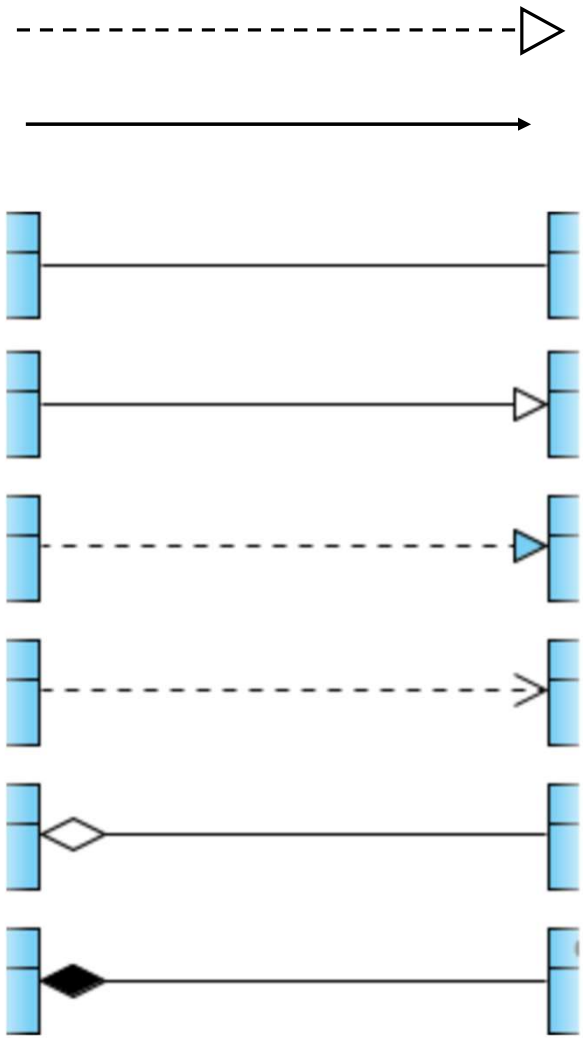
# Attributes

- Attributes



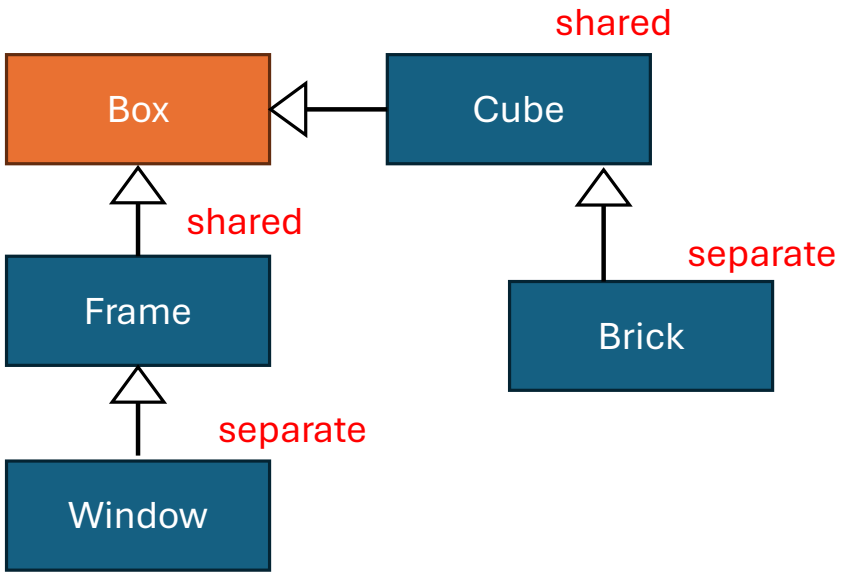
# Arrows

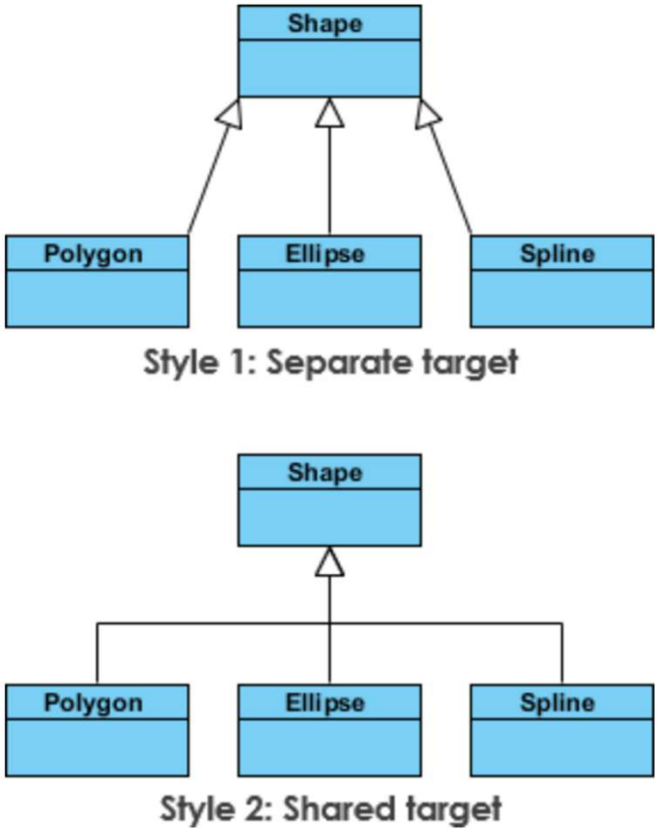
- Connectors



# Targets

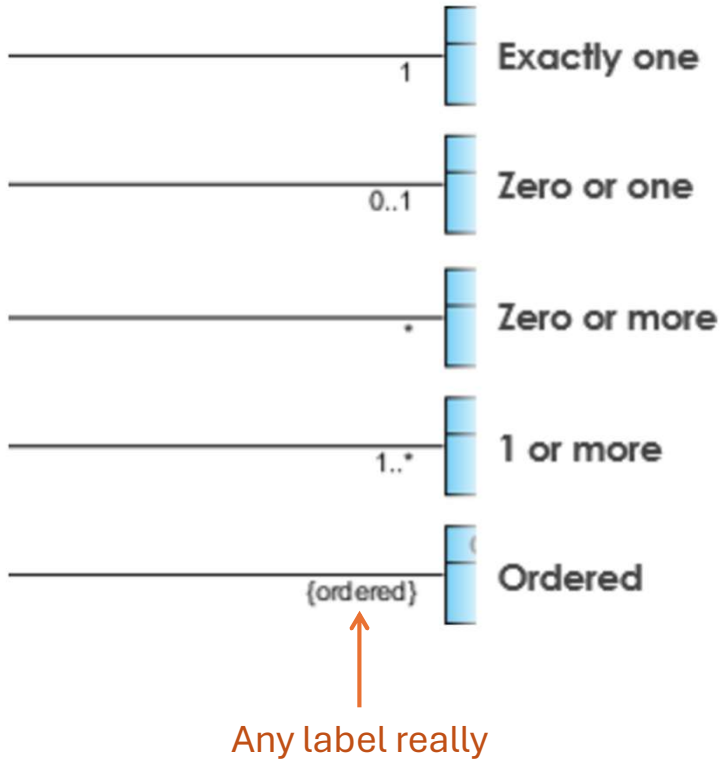
- Inheritance types



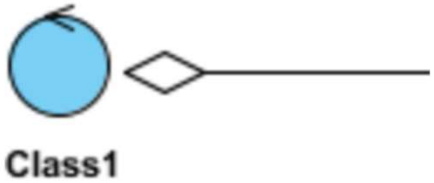
# Number of objects

- Cardinality



# References to objects

- Aggregation




```
Class ABC {
  TYPE reference;

  setter(TYPE p) { reference = p;}
}
```

# References to objects

- Composition

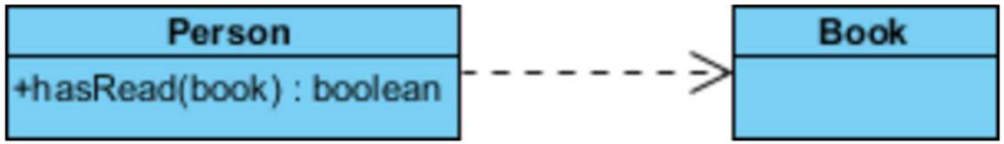



```
Class ABC {
  TYPE reference;

  ABC() {
     reference = new TYPE();
  }
}
```

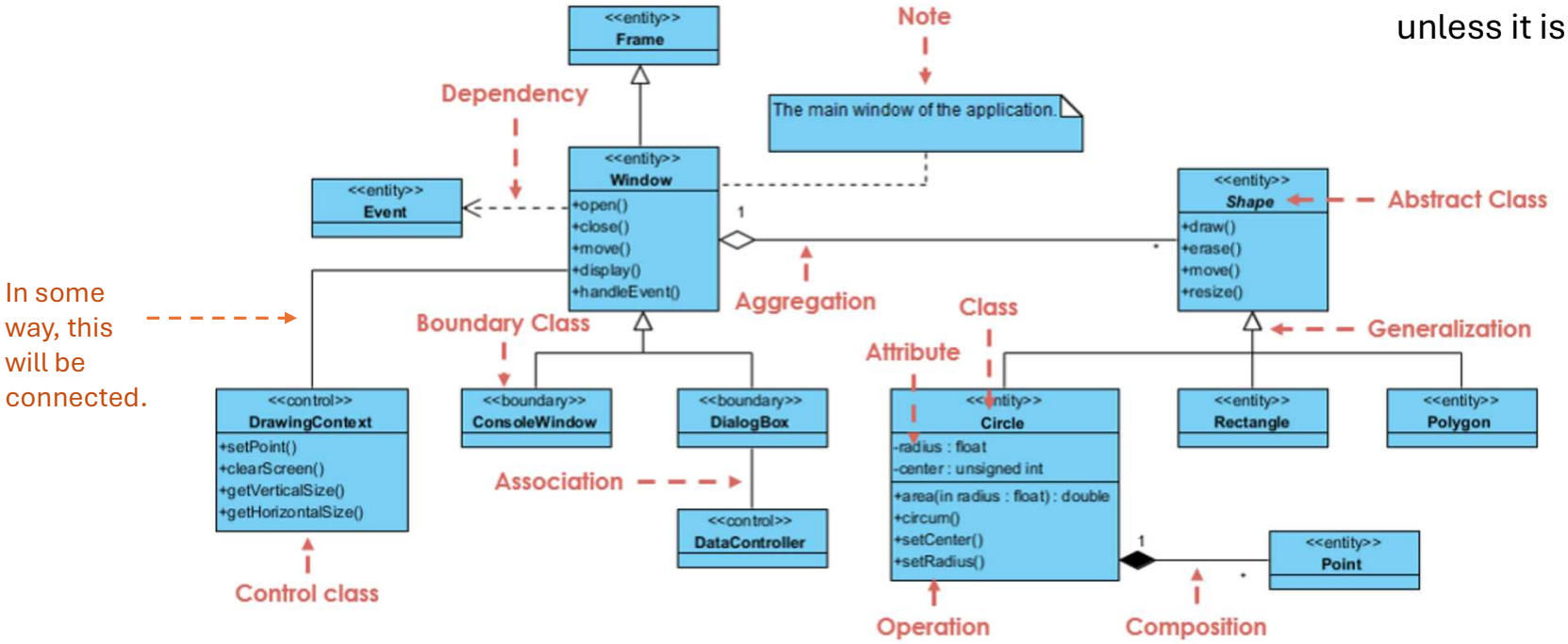
# No reference to object, but there is a local variable

- Dependency



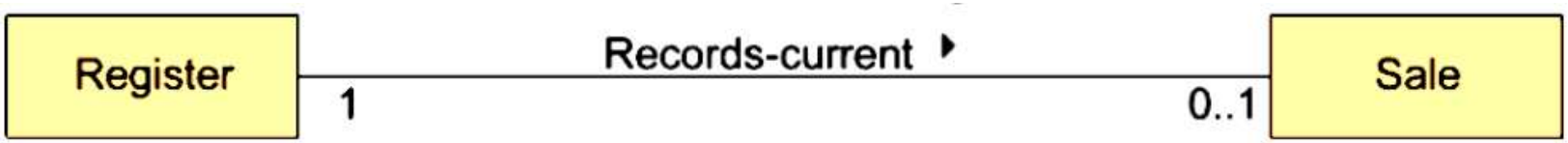
# Domain Model: example 2

**NOTE**: In all submissions to me, you must include the main method and its class. Please don't use the association arrow unless it is necessary.



# Arc labeling

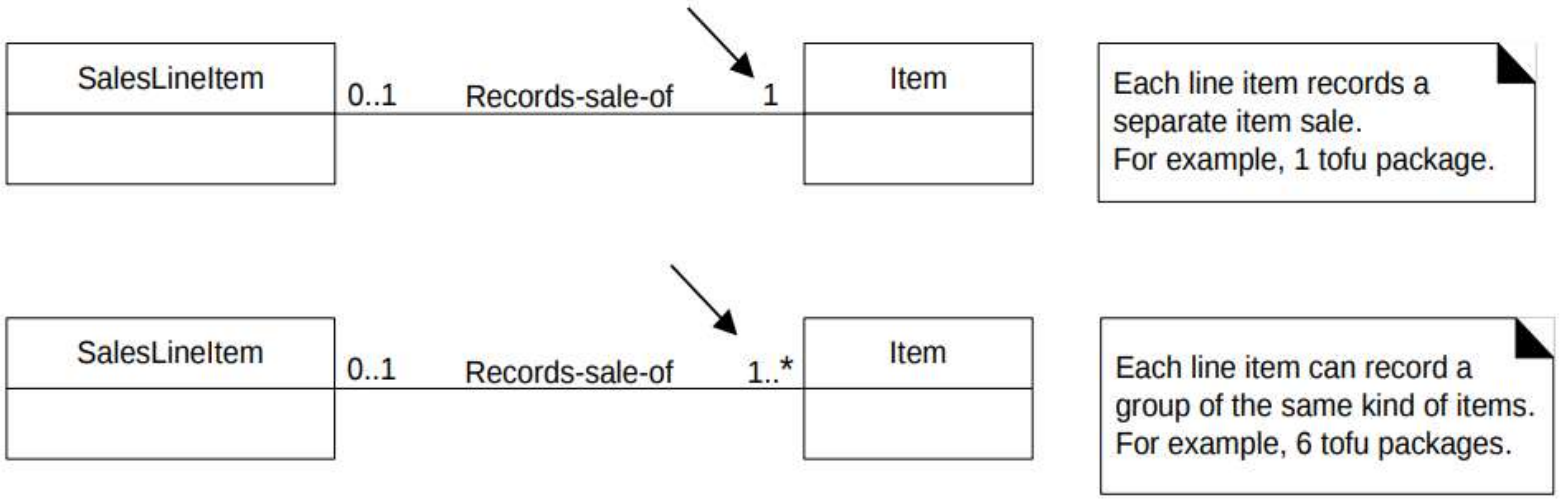
- Labelling & direction



# Arc labeling

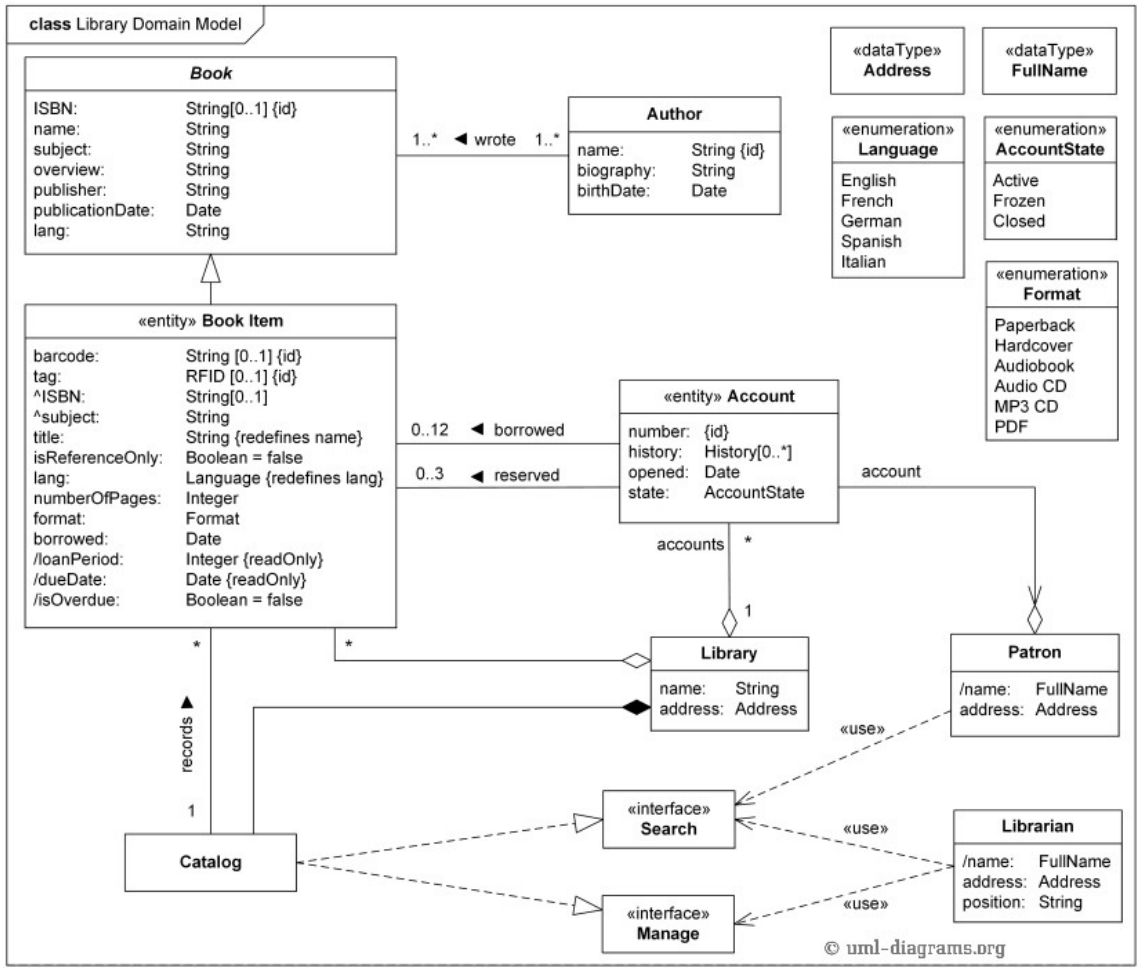
**Notice**: Domain Models are wordier than class diagrams, yet they show fewer class variables and methods.

- Labelling & direction (meanings)



# Example

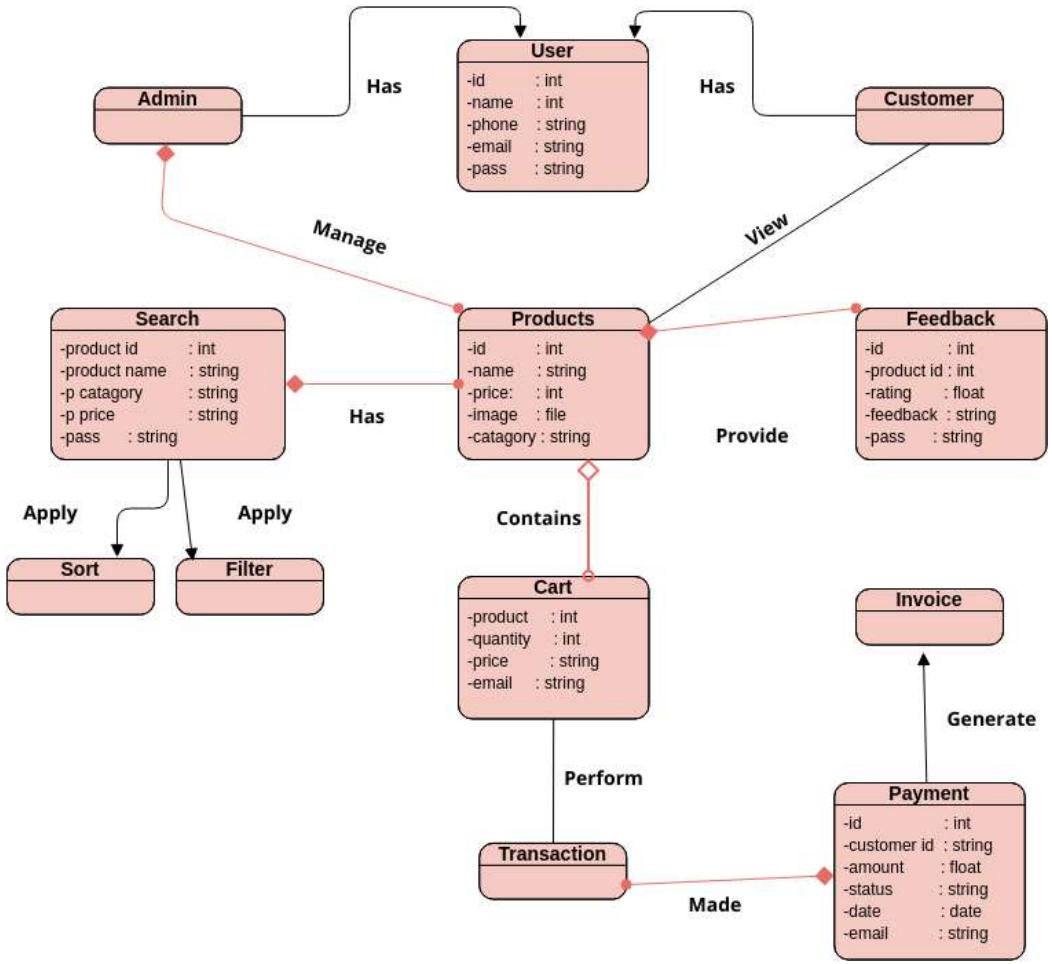
- Directional labels
- Cardinality
- <<comments>>
- Definitions on the side
- UML context box



Recommendation: build the structural picture first (like in a Class diagram), then add flow-related content if required.

# Example

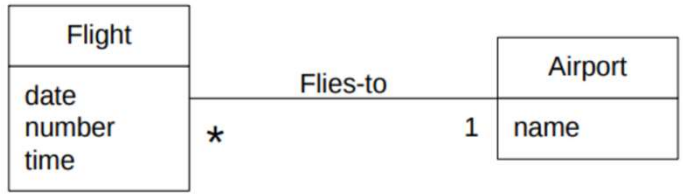
- More label examples that provide both processing information, like “Generate”, and structural information, like “Has” (as in has-a relation).



# Associations

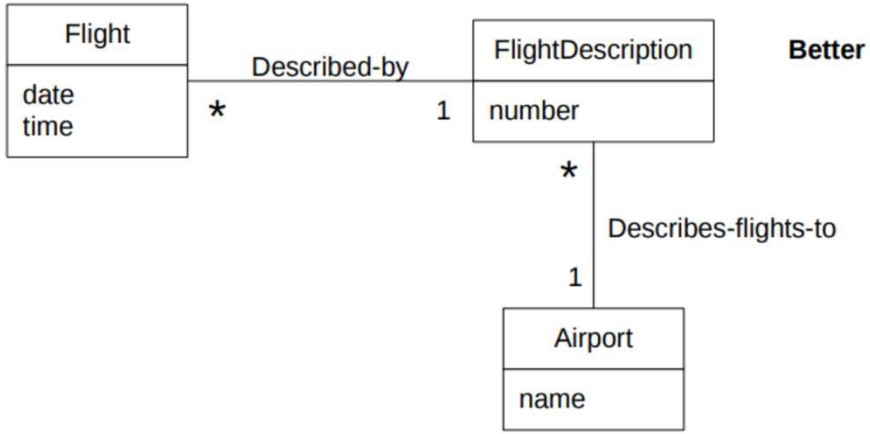
- Operations
- Relationships and data structures

**Notice**: How associations help us build our solution without having to decide on the object.




**Worse**

Associations should not have attributes; if one does have, then chances are it is an object



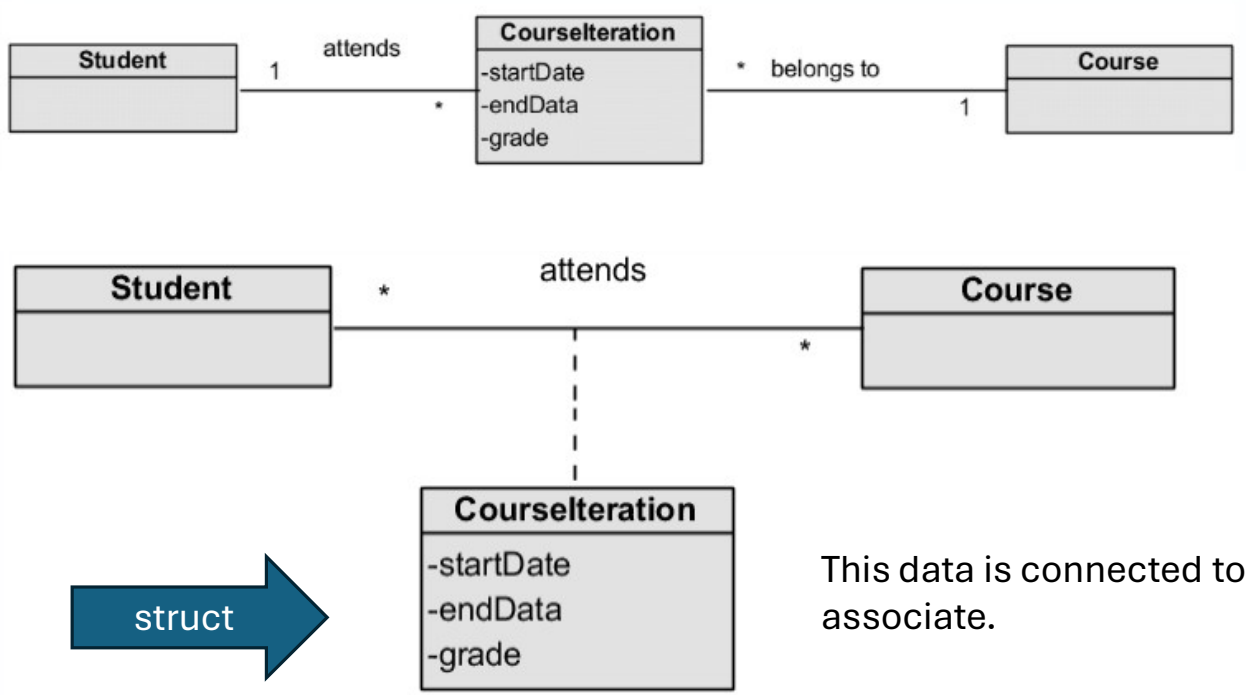

**Better**

Question: coupled by?

# Data structures with objects

- Struct

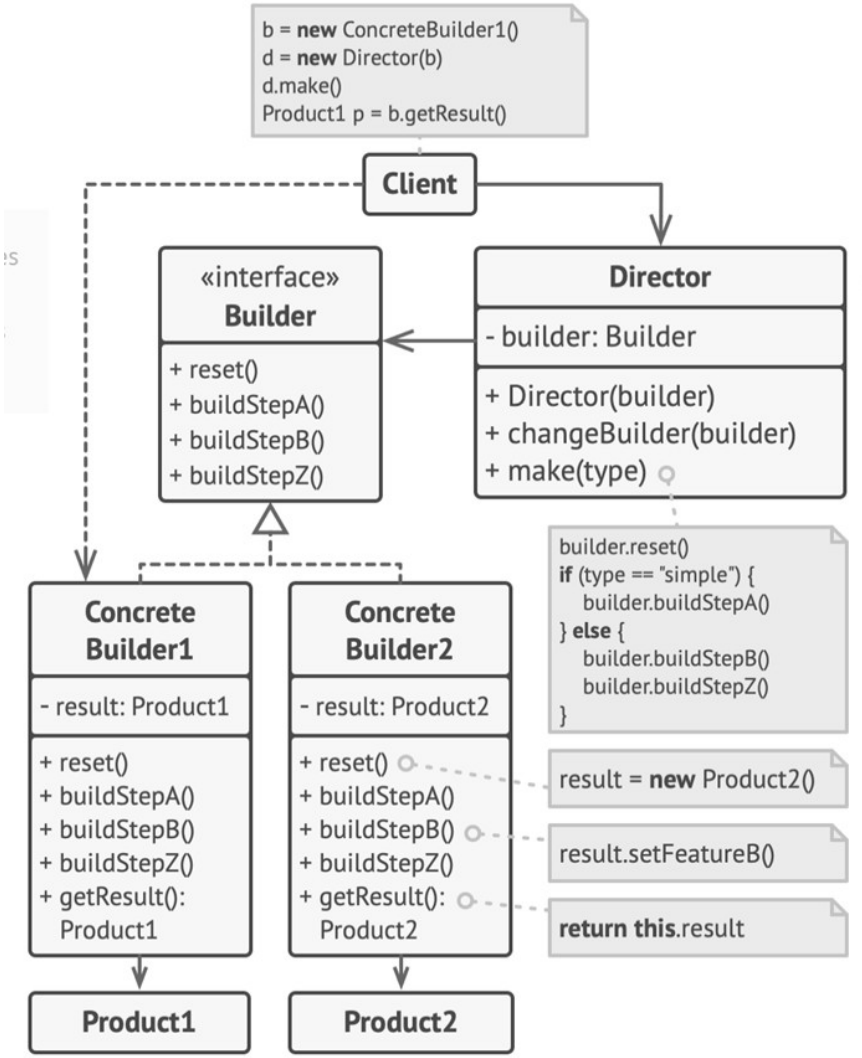
  Or anything similar, like JSON or array, etc.




This data is connected to this associate.

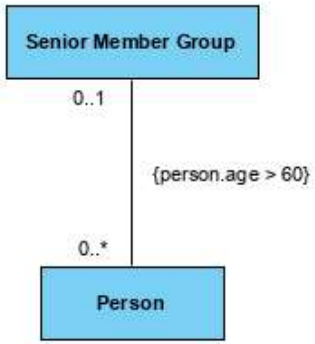
# Defining code

- Notice “Client” is an executing object that calls “Director” and assumes the existence of “Concrete Builder 1”.
- Notice comments with code using dashed connection lines connecting to the method.
  - Critical usage and declaration explanation (typically in pseudo code).



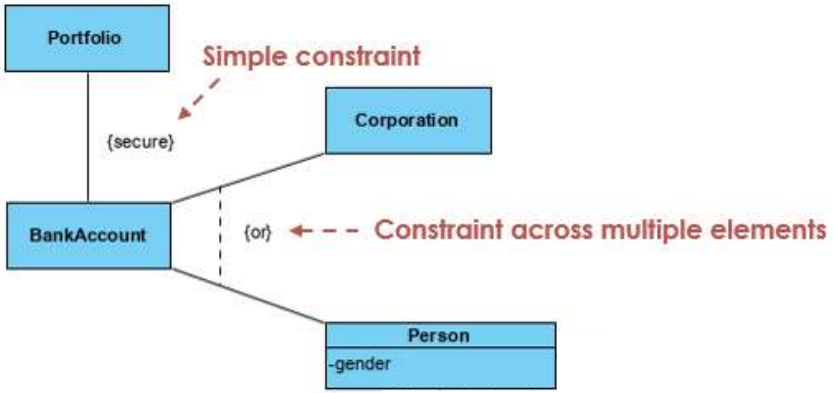
# Constraint labeling

- Constraints (basic)



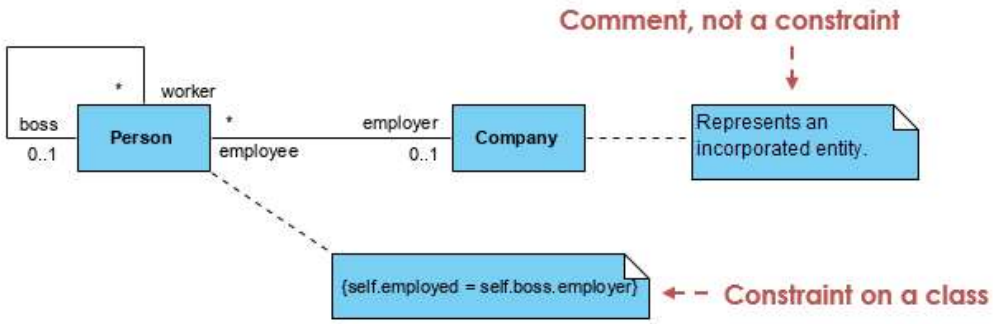
# Constraint labeling

- Constraints



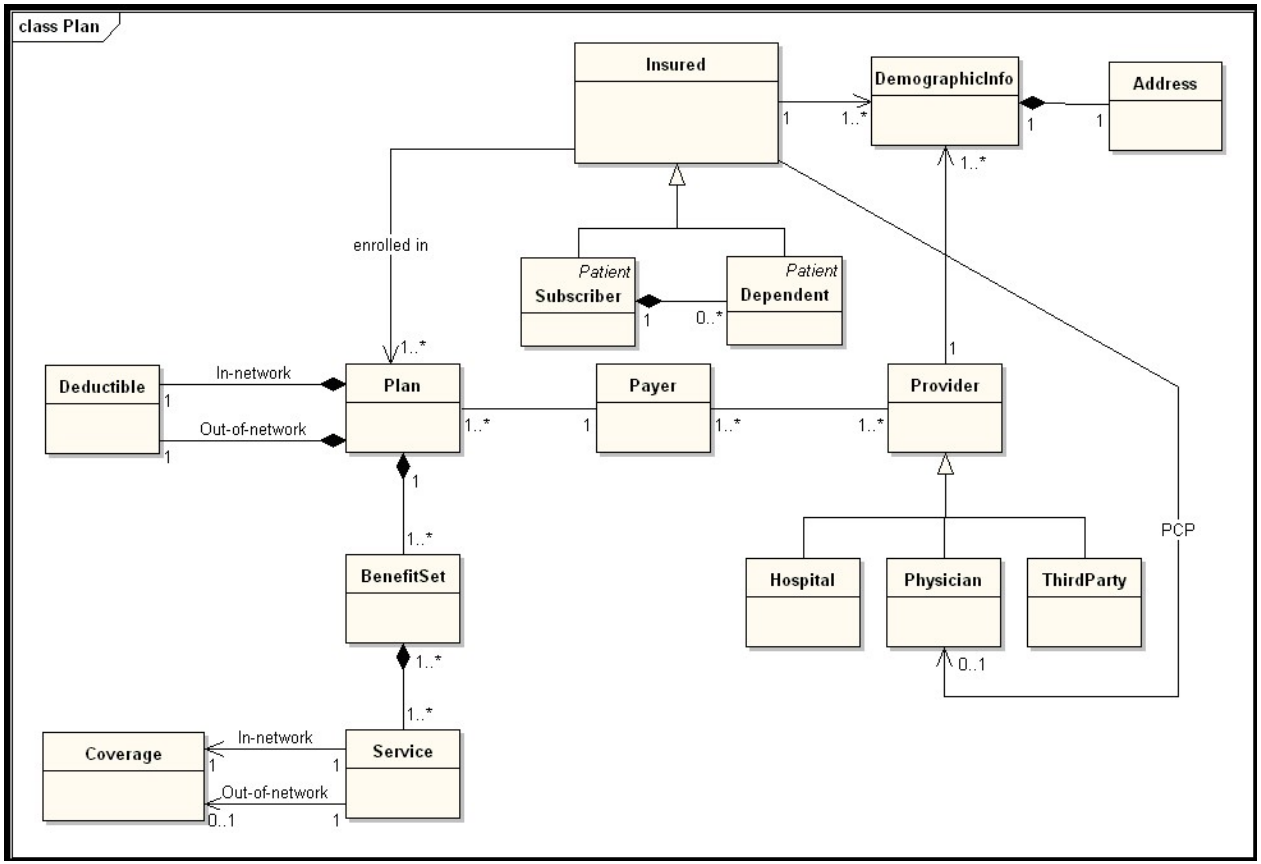
# Constraint labeling

- Constraints (comments)



# Domain Model: Question

- How can we improve?



# Domain Model: Question

- Draw a domain model for this use-case:

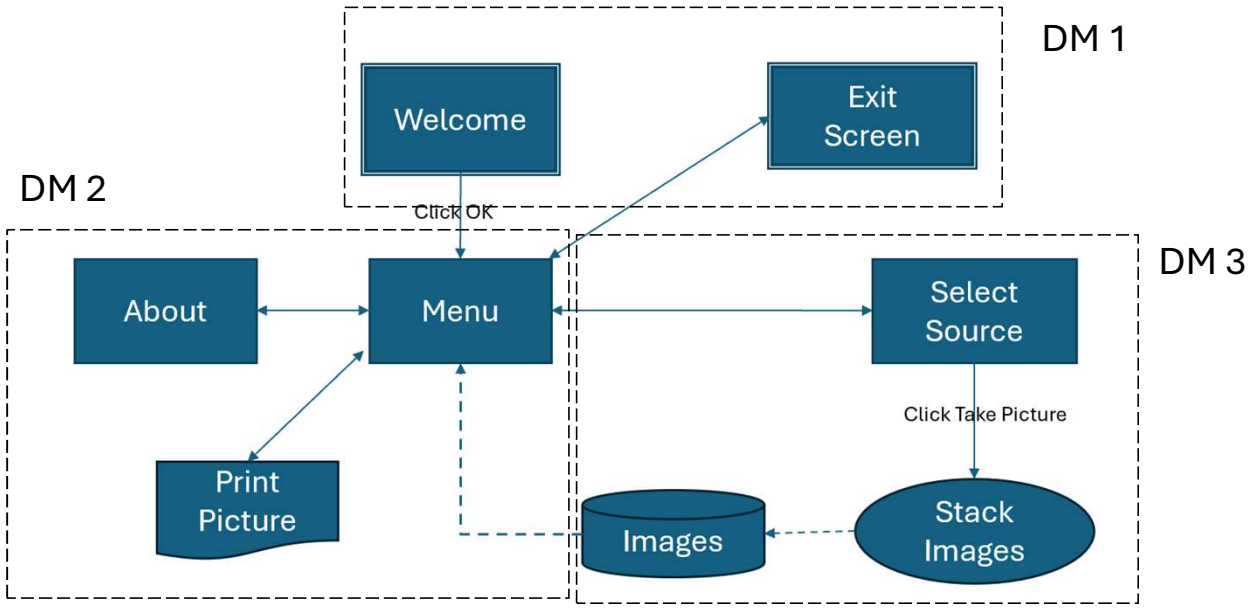
  A Seller wants to sell an item in an online auction setting. The seller posts the item, and people bid on it. At some point, the seller selects the winning Bidder and sends an Invoice for the item to the bidder. The Bidder then pays the Invoice outside of this system. Each item is sorted by a Category.

# Domain Model: Question

What might the Domain model of the Cooking Timer app look like?

# One domain model or many?

- Ideally one, but that is not always possible. Instead:




Use the storyboard to divide the program into components (depicted in this example) that can be labeled (DM i) and placed in their own domain model diagrams.

Format:

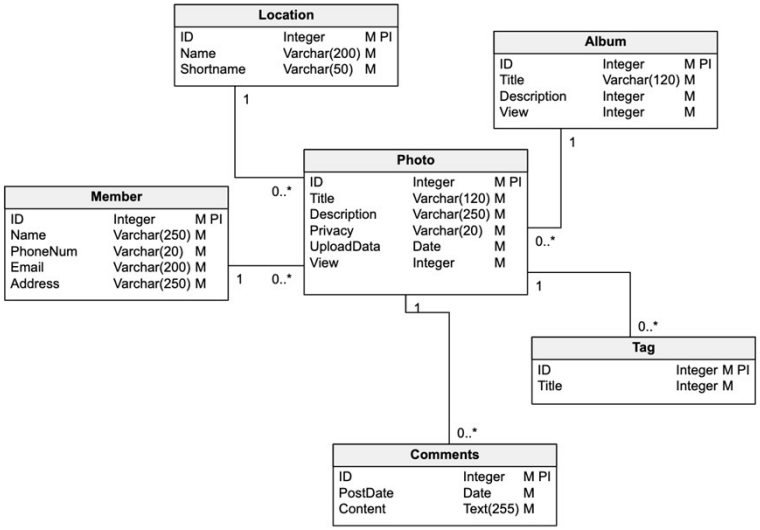
Domain Model 1 – TITLE
Purpose description
How to read the diagram
Domain model diagram

# 4. Database, File and API Definitions

- 4.1 Database Diagrams
- 4.2 Database Schemas
- 4.3 Files
- 4.4 APIs

# Database Construction

- Database Diagram
- Full database structure
  - Validity rules, range, size, type, naming
  - Optimizing for SQL statements
- Indexing
  - For speed
- Important SQL statements
- Important API calls (if applicable)
  - Signature: `URL/Path/Program?var=val&var2=val2`
  - Assumes 3-tiered application



# SQL Syntax

- Table Commands
  - SELECT fields FROM table WHERE condition
  - INSERT INTO table (fields) VALUES (values)
  - DELETE FROM table WHERE condition
  - UPDATE table SET fields=values WHERE condition
- Database Commands
  - CREATE DATABASE name
  - DROP DATABASE name
  - CREATE TABLE name (fieldNames fieldTypes)

# API Syntax

- Signature: `URL/Path/Program?var=val&var2=val2`
- URL
  - IP Address, or
  - http://location
- PATH
  - Directory pathname from HOME or ROOT directory (depending on context)
- Program
  - Name of the program or function invoked
- Parameters
  - Each tuple passed to Program

| API Name |
| --- |
| Signature |
| Variable1 name, type, range |
| Variable2 name, type, range |
| Description |

# Example

- What would an optimal database look like for the School of Computer Science?
  - What should the database be called?
  - What tables are needed?
  - What would the database diagram look like?
  - What are the optimal fields for each table?
  - What SQL operations are needed?
  - If this were a 3-tiered application, what API signatures do we need?

# 4.3 Files

- Configuration files
  - INI files
  - XML files
  - Custom
- Data files
  - CSV files
  - XML files
  - JSON files
  - Custom

Will your program store configuration information in a database or a file?

It depends on the nature of the program. Is the application database-centric, 3-tiered, or has no database?

- If there is no database, then files for data storage are important.
- If you want the user to be able to edit the configurations outside of the program, then files are important.
- If you have a 3-tiered application, then the tiers without a database will probably use files as their way to store data.

# Question

- In the School of Computer Science example, assume it is a web application; would there be non-database storage? If there will be, where will it be located (client-side or server-side)? What would it contain?

# 5. Critical Sections

- 5.1 Risk Analysis
- 5.2 Mitigation Models

WHAT IS A CRITICAL SECTION?

- More than one solution exists, but the client wants a particular solution.
- Order of operations is important.
- Hard to understand what to do.
- Optimization is essential.
- Special requirements impact the algorithm.
- You want it done a particular way.

We realize that Stories, Storyboards, screen mockups, Domain Models, SQL, and APIs are not sufficient to capture the details of information we need for a critical section.

Therefore:

- UML Activity diagram
- UML Sequence diagram
- UML State Machine diagram
- UML Timing diagram
- SQL can also be used in simpler cases

# 5.1 Risk Analysis

- Identification of all the critical sections
  - Best to have a team meeting analyzing the entire program as a brainstorming activity.
- How to represent it:
  - Group critical sections by program location or module.
  - For each critical section, select the best modeling tool.
- For reporting purposes:
  - Begin with a table grouping and listing all critical sections: item, title, description.
  - Followed by a page (or two) presenting the mitigation model for each critical section.

# 5.2 Mitigation Models

- For each identified critical section provide a mitigation algorithm.
- Use either the optional ID number of the critical section name (assuming it is unique), list all the solutions:
  - 5.2.1 Critical section name
    Mitigation algorithm
  - 5.2.2 Critical section name 2
    Mitigation algorithm 2
  - 5.2.3 Critical section name 3
    Mitigation algorithm 3

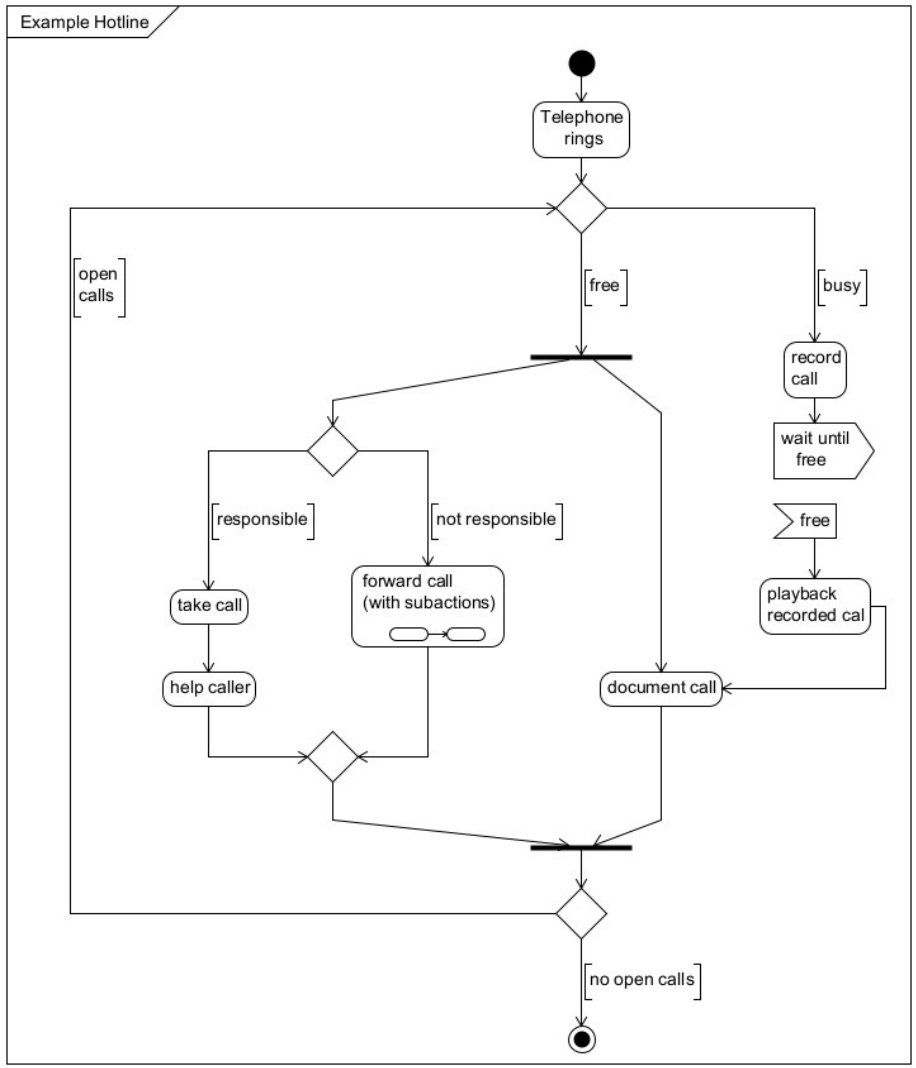
  etc.

Selecting the correct diagram is important

Best for showing the steps in a function (or a few functions)

# UML Activity Diagram

A flowchart of a function



# Question

- What would the UML Activity Diagram be for:

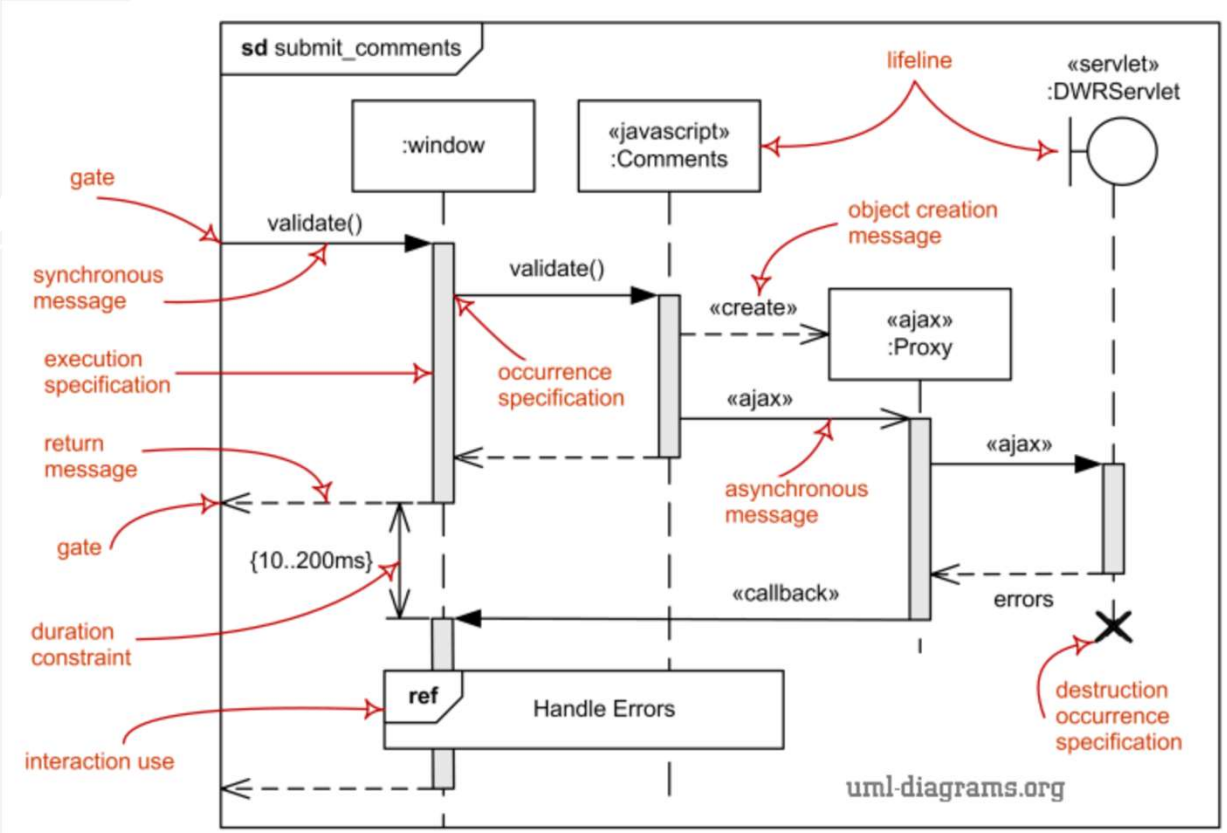
  The algorithm that verifies someone's username and password?

https://www.uml-diagrams.org/sequence-diagrams.html

Best for showing the order of operations between many parts of the program.

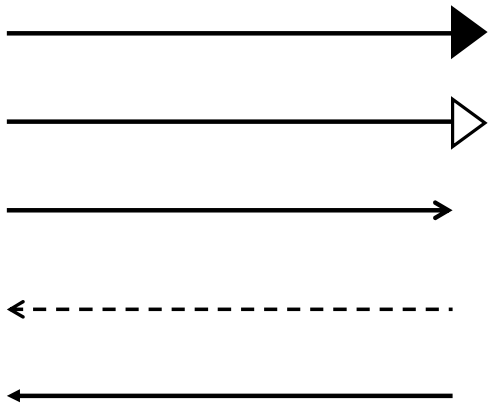
# UML Sequence Diagram

Object-Method Algorithm



# Arrows

- Blocking call
- Parallel call
- Regular call
- Return
- Callbacks



A.K.A. Synchronous call, e.g., semaphore, wait.

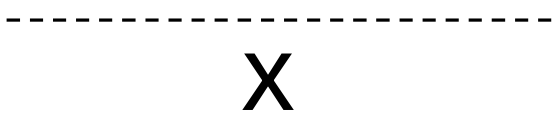
A.K.A. Asynchronous call

Regular function call

Arrows are labeled with the function signature and the return value.

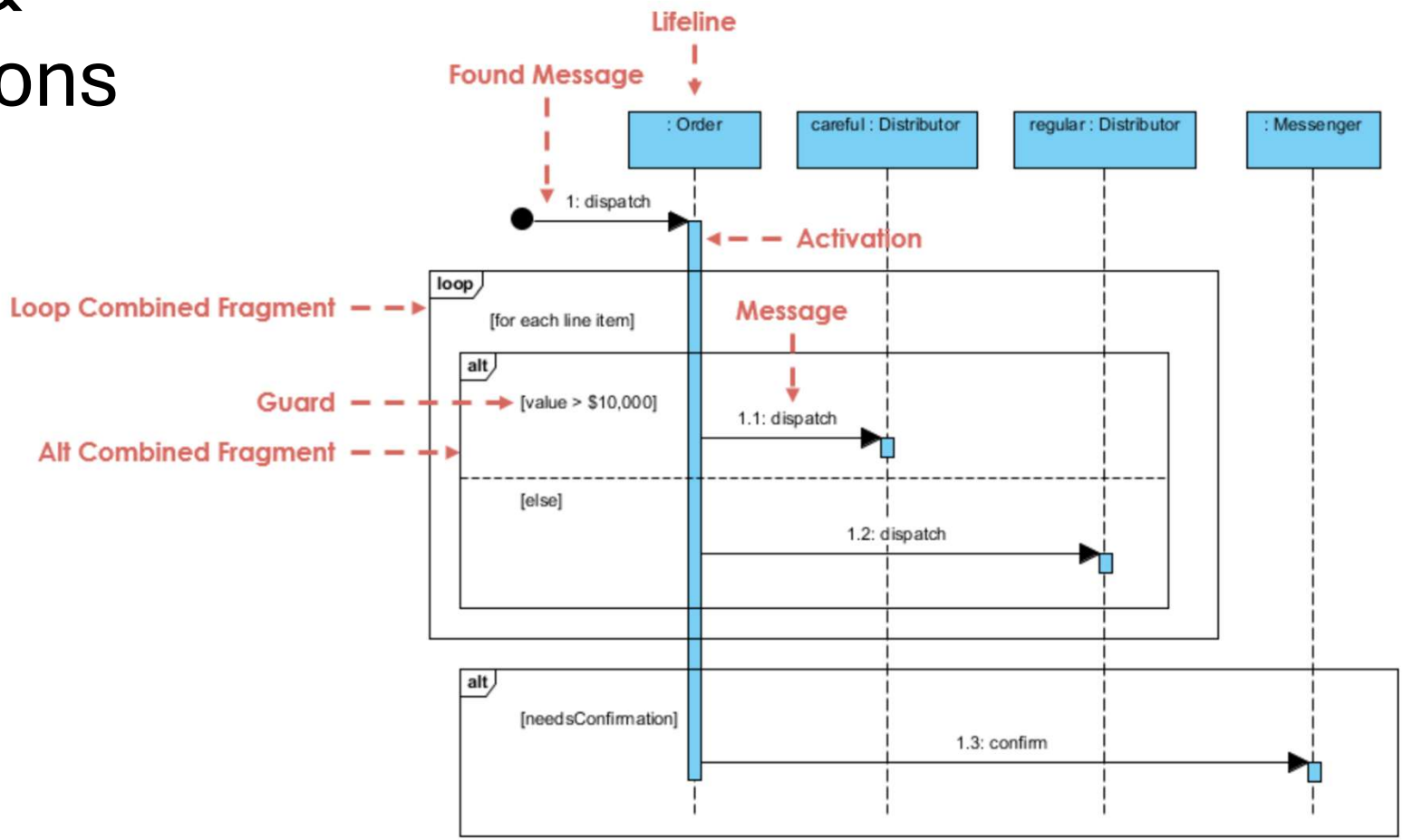
# Structure

- Lifelines
- Destruction
- Focus box
- Arrows into and out of the Focus Box (gates)
- External processes and client programs

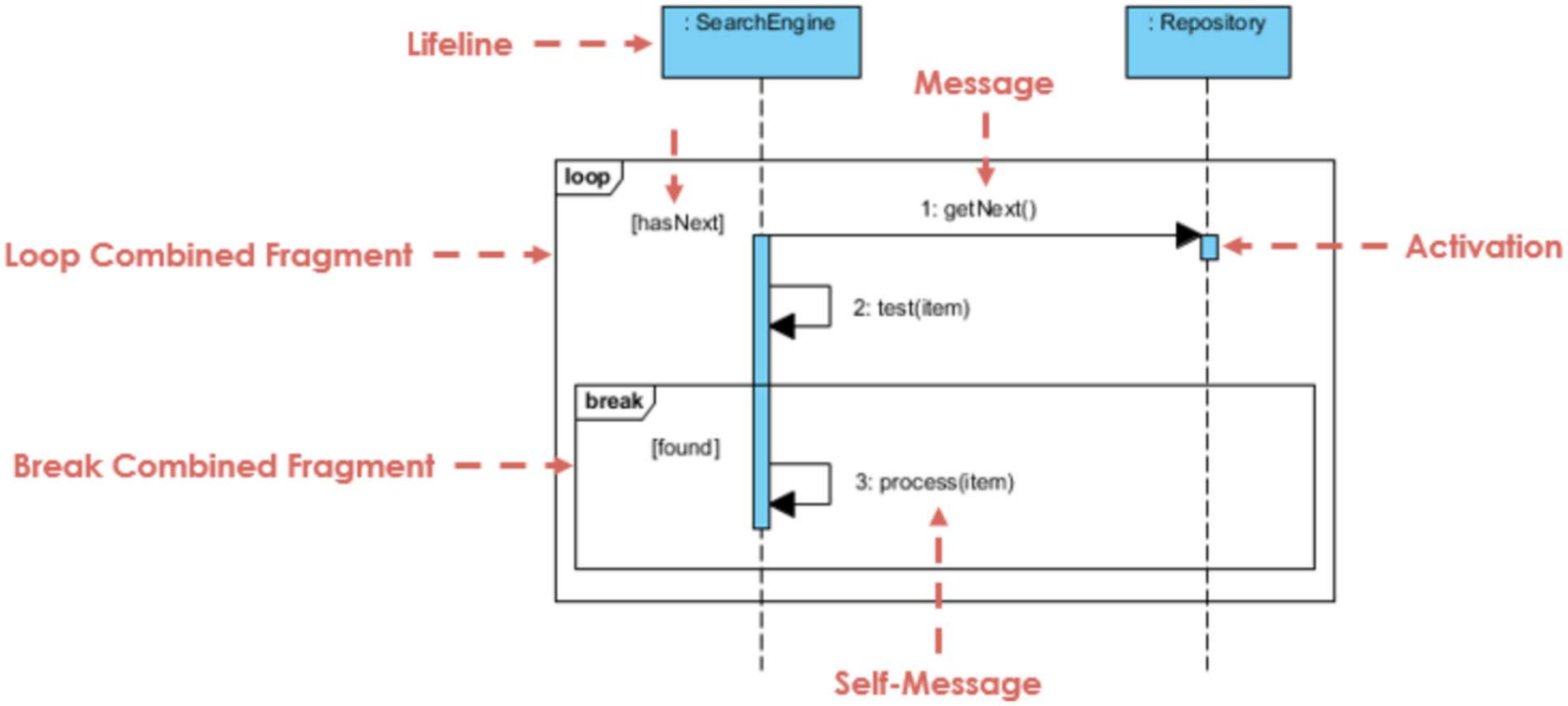




# Loops & Conditions



# Self calls


Lifeline
: SearchEngine
: Repository
Message
loop
[hasNext]
1: getNext()
Loop Combined Fragment
Activation
2: test(item)
break
[found]
Break Combined Fragment
3: process(item)
Self-Message

# Question

- What would the UML Sequence Diagram for:

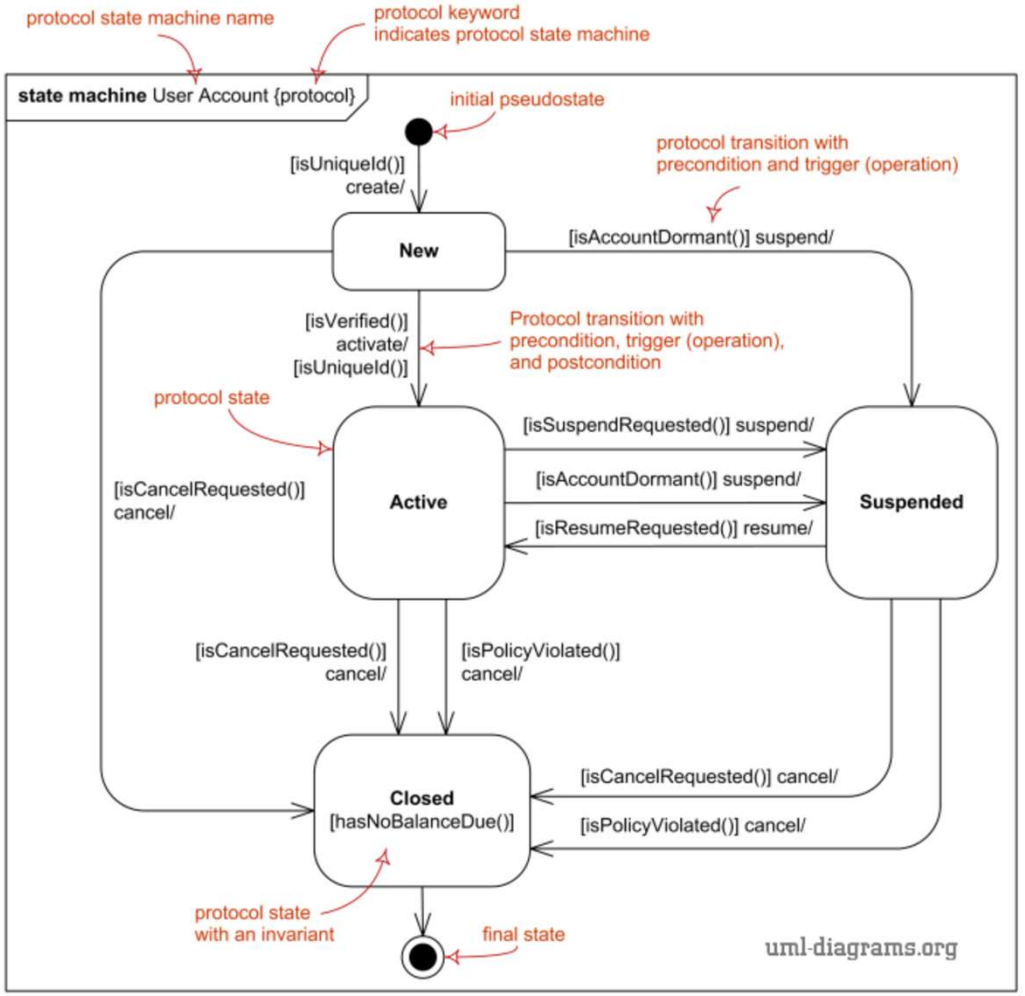
  Cash register app. Scanning all the items into the bill. Then, pay for the bill.

https://www.uml-diagrams.org/protocol-state-machine-diagrams.html

Best for showing event processing (message passing).

# UML State Machine Diagram

When it is easier to talk about event processing.



# Question

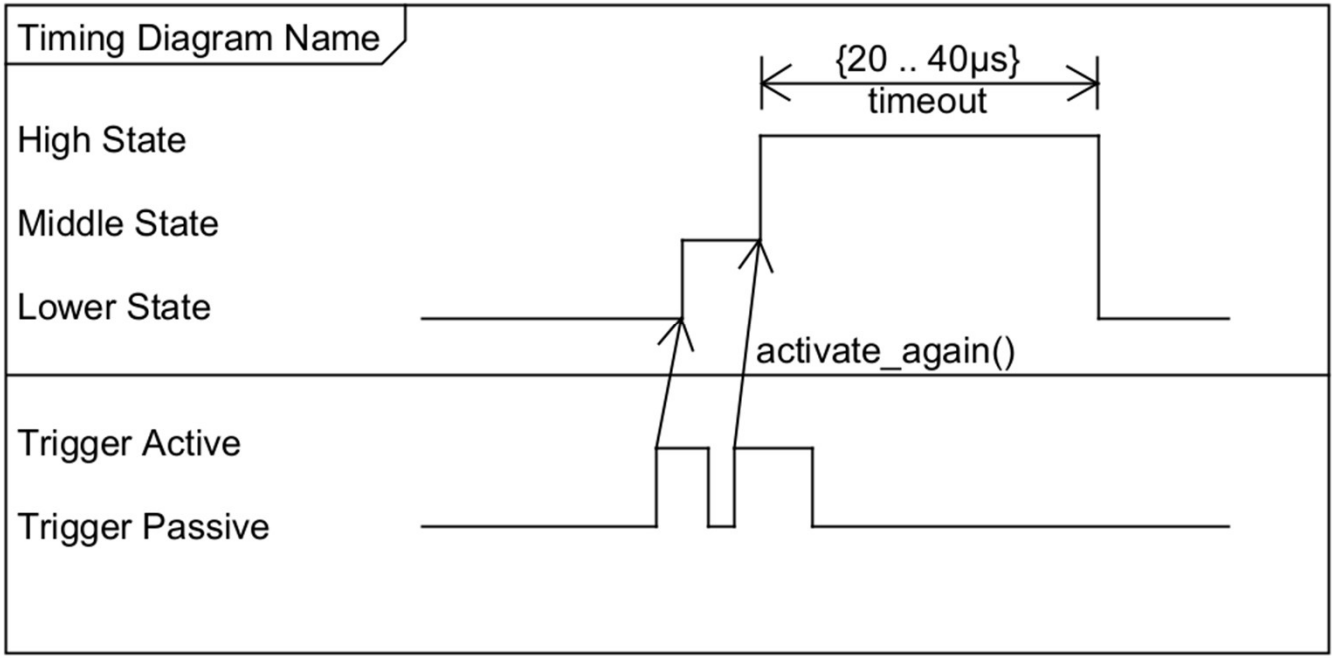
- What would the UML State Machine Diagram for:

  Cooking timer app: set the time, wait for the ring, and then turn off the ringing.

Best for order of operations when timing is critical.

# UML Timing Diagram

When timing is critical.



# Question

- What would the UML Timing Diagram be for:

  Double-click verse single-click in an operating system.

# Question

- What would the UML Timing Diagram be for:

  Describing how packets are received by a network card buffer and copied by the OS into a RAM buffer making it available for the process after it comes out of the Ready Queue.